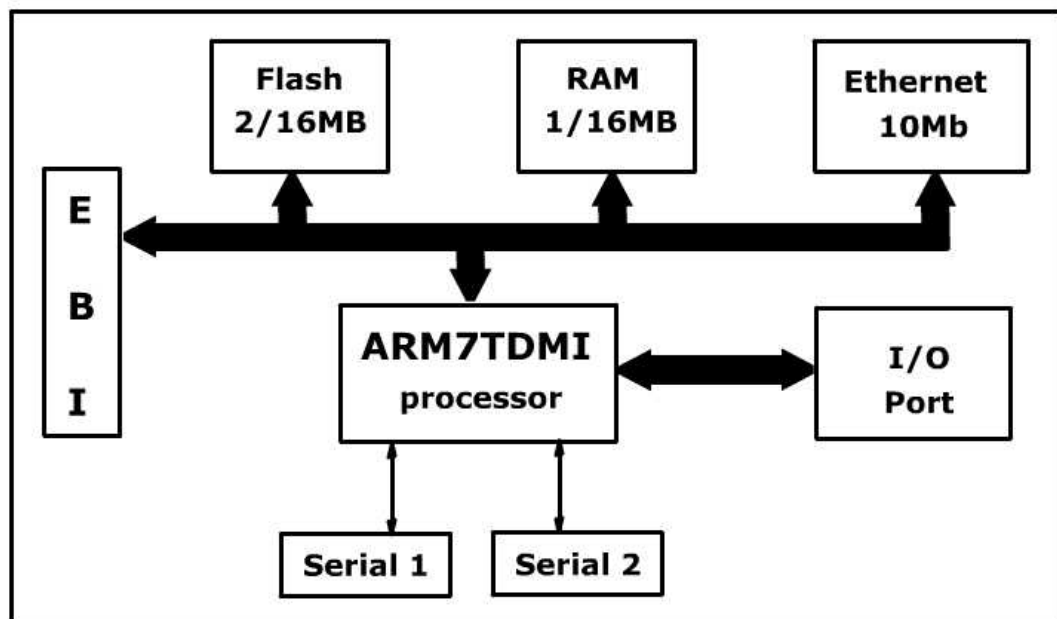


บทที่ 3

การวิเคราะห์และการออกแบบ

3.1 ภาพรวมของโครงการ

โครงการนี้เป็นการพัฒนาบอร์ดควบคุมสำหรับระบบฝังตัว ซึ่งจากการวิเคราะห์โครงสร้างโดยรวมของโครงการจะประกอบด้วย 2 ส่วนหลักๆ คือ ส่วนของฮาร์ดแวร์และซอฟต์แวร์ โดยส่วนของฮาร์ดแวร์เป็นการพัฒนาบอร์ดควบคุมที่ใช้กับระบบฝังตัว โดยโครงสร้างจะประกอบด้วย ไมโครโปรเซสเซอร์ ARM7TDMI , หน่วยความจำแฟลช 2 เมกกะไบต์ หน่วยความจำแรม 1 เมกกะไบต์ อีเทอร์เน็ตคอลโทรลเลอร์ความเร็ว 10 เมกกะบิตต่อวินาที , บัสภายนอก(External Bus Interface) สำหรับต่อขยายหน่วยความจำที่สามารถที่จะขยายสูงสุดได้ถึง 64 เมกกะไบต์, พอร์ตขนานที่ใช้สำหรับติดต่อกับอุปกรณ์ภายนอก , พอร์ตอนุกรมจำนวน 2 พอร์ตโดยพอร์ตนึงสำหรับติดต่อกับโฮสต์(Host System)และอีกพอร์ตนึงสำหรับใช้งานทั่วไป ดังรูป



รูปที่ 3.1 โครงสร้างของบอร์ดควบคุม

ในส่วน of ซอฟต์แวร์จะเป็นการพัฒนาเครื่องมือที่ใช้ในการโปรแกรมมิ่งหน่วยความจำแฟลช(Onboard-Programming) และระบบปฏิบัติการที่ใช้บนบอร์ดควบคุม คือ อีคอส(eCos-Embedded Configuration Operating System)

3.2 ส่วนประกอบทางด้านฮาร์ดแวร์

ในการวิเคราะห์ส่วนประกอบทางด้านฮาร์ดแวร์ เป็นการวิเคราะห์ถึงส่วนที่จำเป็นในการเลือกอุปกรณ์ที่จะนำมาใช้เป็นส่วนประกอบของบอร์ด ให้มีความเหมาะสมและเป็นไปตามเป้าหมายของโครงการ โดยในการเลือกจะพิจารณาถึงลักษณะของการเชื่อมต่อ ความสามารถและคุณสมบัติที่ต้องการ รวมทั้งต้องพิจารณาถึงความยากง่ายในการนำมาใช้

3.2.1 เปรียบเทียบไมโครคอนโทรลเลอร์

การเลือกโปรเซสเซอร์จะเลือกเป็นไมโครคอนโทรลเลอร์ มากกว่าที่จะเป็นไมโครโปรเซสเซอร์เพราะได้รวมเอาอุปกรณ์พื้นฐานที่จำเป็นสำหรับบอร์ดควบคุมไว้ ทำให้บอร์ดควบคุมมีขนาดเล็กลง

ตารางที่ 3.1 การเปรียบเทียบคุณสมบัติไมโครคอนโทรลเลอร์ AT91-4x , ElanSC4xx , Motorola 68K

คุณสมบัติ/ซีพียู	Elan TM SC4xx	AT91-4x	Integrated 68K MC683xx
ผู้ผลิต(ตระกูล)	AMD(486SX)	Atmel(ARM7TDMI)	Motorola(MC68K)
ชื่ออุปกรณ์	ElanSC400 ElanSC410	AT91M40800 AT91R40807	MC68332 MC68334
สถาปัตยกรรม	CISC	RISC	CISC
ขนาดบัสภายใน/นอก	32/32,16,8	32/16,8	32/32
DRAM Controller	ใช่	ไม่ใช่	ไม่ใช่
MIPS/Bus Frequency	63.5@100Mhz/33,66,100	36@40MHz/40 MAX	9.8@25MHZ
Interfaces/ Address Space	ISA Bus,VL Bus/ 4GB	External Bus/ 64MB	Async SCI and queued sync (SPI) / 4GB
Peripherals Interfaces	Interrupt Controllers UART, Keyboard IF, 32 Gen Purpose I/O, PCMCIA (400), LCD Controller (400)	Timers, Watchdog, Interrupt Controller, Power Management, Oscillator, PLL	Timer , Watchdog , Bus Monitor, Periodic Interrupt Timer, Background Debug Mode, PIT, TPU TimeBase Counter
Vdd (core I/O) Power Dissipation	750mW@33MHz/ 5V 1.8W @ 100MHz	46mW/MHz @ 3.3V	125-700mW @ 5V and 16-25MHz
ราคาประมาณ	452 – 1654 บาท	633 – 856 บาท	966-1850 บาท

เนื่องจากโครงการนี้ต้องการที่จะพัฒนาบอร์ดควบคุมที่สามารถรองรับการประมวลผลทางด้านเครือข่ายคอมพิวเตอร์ บวกกับความสามารถในการทำ multitasking ของระบบปฏิบัติการซึ่งต้องการโปรเซสเซอร์ที่มีความเร็วในการประมวลผลสูง แต่ก็ยังคงพิจารณาถึงความเหมาะสมกับระบบฝังตัวขนาดกลาง การสิ้นเปลืองพลังงาน และราคาก็เป็นสิ่งจำเป็น ที่ต้องคำนึงถึง ตารางข้างบนเป็นการเปรียบเทียบไมโครคอนโทรลเลอร์ที่มีคุณสมบัติดังได้กล่าวไปแล้ว และมีการใช้งานอย่างกว้างขวาง

3.2.1.1 ไมโครคอนโทรลเลอร์ Elan TM SC4xx Processor

เป็นชิพยูทระกูลอินเทล 486DX ขนาดโปรเซสเซอร์ 32 บิต ใช้พลังงาน 5 โวลต์ ทำงานที่ 33,66,100 MHz มีสถาปัตยกรรมแบบ CISC สนับสนุนบัสภายนอก (external data bus) ทั้ง 8,16 และ 32 บิต มีความสามารถใช้ได้ร่วมกันได้กับ 8086, 80286 และโปรเซสเซอร์ตระกูลอินเทล 386 และสามารถเอ็กซ์คิวทิวหลายๆ คำสั่งด้วยสัญญาณคล็อกเพียงลูกเดียว มีการทำงานแบบไปป์ไลน์แบบ fetching, decoding, address translation และ execution ข้อดีของไมโครคอนโทรลเลอร์เบอร์นี้คือรวมอุปกรณ์ที่มีอยู่ในพีซี (Personal Computer) เกือบทั้งหมดเอาไว้ ทำให้มีจำนวนขาและแพ็คเกจที่ค่อนข้างยากในการประกอบเป็นบอร์ดควบคุม เบอร์ที่สนใจคือ ElanSC400 เพราะรวมเอาวงจรจับแอดเดรสได้เข้าไว้ด้วย ทำงานที่ ความเร็วคล็อก 33 MHz สามารถต่อหน่วยความจำแบบดีเรม (DRAM) ได้

3.2.1.2 ATMEAL ARM7TDMI (AT91-4x)

AT91M40800 เป็นไมโครคอนโทรลเลอร์ 32 บิต และมีโปรเซสเซอร์ภายในเป็น ARM7TDMI ที่มีสถาปัตยกรรมแบบ RISC ใช้พลังงาน 3.3 โวลต์ ทำงานที่ 33 MHz มีชุดคำสั่งเป็น 16 บิต ใช้พลังงานต่ำ สามารถเอ็กซ์คิวทิวหลายๆ คำสั่งด้วยสัญญาณคล็อกเพียงลูกเดียว มีการกำหนดการทำงานของอุปกรณ์ที่ดีสำหรับรองรับแอปพลิเคชันแบบเรียลไทม์ (real-time) และมีการจัดระดับความสำคัญ (Priority) ของอินเตอร์รัพท์เวกเตอร์ 8 ระดับ มีพอร์ตไว้สำหรับต่อขยายหน่วยความจำ (EBI = external bus interface) เหมาะสำหรับที่จะนำไปใช้พัฒนาระบบฝังตัว เนื่องจากเป็นโปรเซสเซอร์ที่เลือกใช้จะได้อีกในรายละเอียดต่อไป

3.2.1.3 Integrated 68K MC683xx

เป็นไมโครโปรเซสเซอร์ 32 บิต ซึ่งอยู่ในตระกูลไมโครโปรเซสเซอร์ 68000 ของ Motorola ใช้เทคโนโลยี VLSI (Very Large Scale Integration) คาปาซิเตอร์ 32 บิต และ แอดเดรส 32 บิต สนับสนุนภาษาระดับสูง, แอสเซมบลี, มีการติดต่อภายในไมโครโปรเซสเซอร์เป็นแบบขนาน สามารถเอ็กซ์คิวทิวได้หลายคำสั่งพร้อมกัน มีโครงสร้างบัสเป็นแบบบัสโครนัส โปรเซสเซอร์สามารถขยายขนาดของบัสได้ เป็นไมโครโปรเซสเซอร์ที่ออกแบบมาสำหรับระบบฝังตัว ข้อดีของไมโครคอนโทรลเลอร์เบอร์นี้คือรวมอุปกรณ์ต่างเข้าไว้หลายตัวด้วยกัน แต่ก็ยังมีความเร็วไม่ค่อยดีนักเมื่อเปรียบเทียบกับโปรเซสเซอร์ที่มีอยู่ในตาราง อีกทั้งราคาก็ยังแพงอยู่มาก

3.2.2 เหตุผลที่ใช้ในการเลือกไมโครโปรเซสเซอร์

ไมโครโปรเซสเซอร์ที่มีอยู่อย่างมากมายนั้น ต่างก็ได้ถูกออกแบบมาเพื่อให้ใช้งานในด้านต่างๆ กัน และมีความสามารถในการทำงานต่างกันด้วย หลักที่ใช้ในการพิจารณามีหลายอย่างไม่ว่าจะเป็น ด้านประสิทธิภาพการทำงาน การใช้พลังงาน ความยากง่ายในการพัฒนา รวมไปถึงการมีเครื่องมือสนับสนุนที่ใช้ในการพัฒนา และราคาของตัวไมโครโปรเซสเซอร์เมื่อเทียบกับการนำมาใช้งาน เนื่องจากโครงการนี้เป็นการพัฒนาบอร์ดที่ใช้เป็นต้นแบบสำหรับระบบฝังตัว จึงทำให้มีข้อพิจารณาในเบื้องต้นในการคัดเลือกไมโครโปรเซสเซอร์ ดังนี้

ตารางที่ 3.2 การเปรียบเทียบไมโครคอนโทรลเลอร์ AT91M40800 , ElanSC400 ,MC68332

ข้อพิจารณา	AT91M40800	ElanSC400	MC68332
ขนาด (บิต)	32	32	32
สถาปัตยกรรม	RISC	CISC	CISC
พลังงานที่ใช้(วัตต์)	0.15	1.14	1.2
ความเร็ว	30MIPS@33MHz	7.6MIPS@33MHz	9.8MIPS@25MHZ
อุปกรณ์พื้นฐาน Timers, Watchdog, Interrupt Controller, Serial Port	มี	มี	มี
พอร์ต JTAG	มี	มี	มี
พื้นที่หน่วยความจำ	64MB	4GB	4G
ราคา (บาท)	633	1,386	966

เนื่องจากปัจจุบันนี้การทำงานของชิปและอุปกรณ์โดยทั่วไปจะสนับสนุนการทำงานที่ขนาด 32 บิตมากขึ้นราคาไม่แตกต่างกับโปรเซสเซอร์มากนัก และความต้องการของผู้ใช้งานที่ต้องการความเร็วมากขึ้น จึงให้ความสำคัญกับโปรเซสเซอร์ขนาด 32 บิต เนื่องจากต้องการให้ง่ายในการออกแบบจึงต้องหาอุปกรณ์ที่รวมเอาสิ่งจำเป็นพื้นฐานเอาไว้ การสิ้นเปลืองพลังงานก็เป็นสิ่งสำคัญ นอกจากนี้อุปกรณ์ในระบบฝังตัวมักจะไม่มีความเล็กจึงจำเป็นต้องมีสิ่งที่ใช้ในการตรวจสอบข้อผิดพลาด การทำ On-Board Programming เช่น พอร์ต JTAG(Joint Test Action Group)

ราคาก็เป็นอีกสิ่งหนึ่งที่ใช้เป็นปัจจัยในการพิจารณา จากตารางด้านล่างจะเห็นได้ว่า ARM7TDMI เป็นไมโครคอนโทรลเลอร์ ที่มีราคาถูกกว่าตัวอื่น ๆ และเมื่อได้พิจารณาประสิทธิภาพโดยรวมกับการนำมาใช้งานสำหรับโครงการนี้ ARM7TDMI ก็เป็นไมโครคอนโทรลเลอร์ที่น่าจะนำมาใช้มากที่สุด

3.3 องค์ประกอบของบอร์ด

ซึ่งบอร์ดที่พัฒนาขึ้นจะใช้ไมโครคอนโทรลเลอร์ AT91M40800 ซึ่งเป็นของบริษัท ATMEL ที่มี ไมโครโปรเซสเซอร์เป็น ARM7TDMI

3.3.1 ส่วนประกอบของบอร์ด

- ไมโครคอนโทรลเลอร์ AT91M40800 โดยมีโปรเซสเซอร์ภายในเป็น ARM7TDMI
- ตัวสร้างสัญญาณนาฬิกา (Clock Generator)
- SRAM 1 เมกกะไบต์ สามารถต่อขยายได้ 16 เมกกะไบต์
- Flash Memory 2 เมกกะไบต์ สามารถต่อขยายได้ 16 เมกกะไบต์
- EBI บัส สำหรับต่อขยายหน่วยความจำ
- พอร์ตขนานที่ใช้สำหรับติดต่อกับอุปกรณ์ภายนอก
- ซีเรียลพอร์ตจำนวน 2 พอร์ต
- พอร์ตขนาน
- ปุ่มรีเซ็ต

3.3.2 ไมโครโปรเซสเซอร์ ARM7TDMI

เป็นไมโครโปรเซสเซอร์ขนาด 32 บิต ตระกูล ARM (Advanced RISC Machines) มีประสิทธิภาพสูง สิ้นเปลืองพลังงานน้อย ราคาถูก

สถาปัตยกรรม ARM ใช้หลักการของ RISC (Reduced Instruction Set Computer) โดยมีชุดคำสั่งและวิธีการตีโค้ด (Decode) ที่ง่ายกว่า CISC (Complex Instruction Set Computer) ทำให้คำสั่งเหล่านี้มี Throughput ที่สูง และเหมือนกับว่ามีการตอบรับการอินเทอร์รัพท์ในลักษณะของเรียลไทม์จากชิปที่มีขนาดเล็กและทำให้คุ้มค่าในการลงทุน

มีการนำหลักการไปป์ไลน์ (Pipeline) ไปใช้กับทุกส่วนของโปรเซสเซอร์และกับระบบหน่วยความจำซึ่งสามารถทำการโอเปอเรชันได้อย่างต่อเนื่อง อาทิเช่น เมื่อมีคำสั่งหนึ่งถูกเอ็กซีคิวต์ โปรเซสเซอร์จะเริ่มทำการตีโค้ดและคำสั่งต่อไปจะถูกดึงมาจากหน่วยความจำ

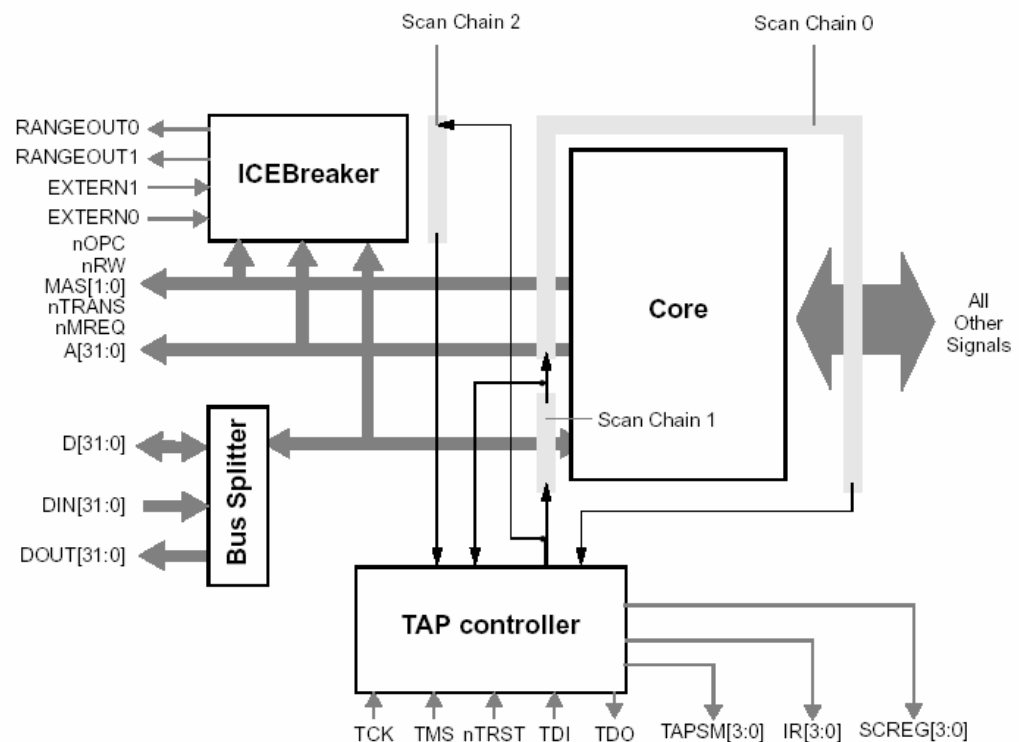
ในการเชื่อมต่อกับหน่วยความจำใน ARM ถูกออกแบบให้ใช้งานได้อย่างมีประสิทธิภาพกับระบบหน่วยความจำที่ราคาไม่สูง สัญญาณควบคุมที่ถูกควบคุมโดยไปป์ไลน์นั้นใช้ตรรกะมาตรฐานแบบพลังงานต่ำและรูปแบบของสัญญาณ ที่สามารถใช้ได้กับโหมดการเข้าถึงแบบเร็วของหน่วยความจำมาตรฐานในอุตสาหกรรม

ส่วนคุณสมบัติอื่นๆเพิ่มเติมนอกจากนี้ได้แก่

- ดีบั๊กแบบบนชิปโดย ICEBreaker
- ไปป์ไลน์ 3 สถานะ
- โปรเซสเซอร์ RISC 32 บิต เป็นสถาปัตยกรรมแบบ VON Neuman load/store ซึ่งมีบัส สำหรับแอดเดรสและ ข้อมูลสำหรับชุดคำสั่งและชุดข้อมูล

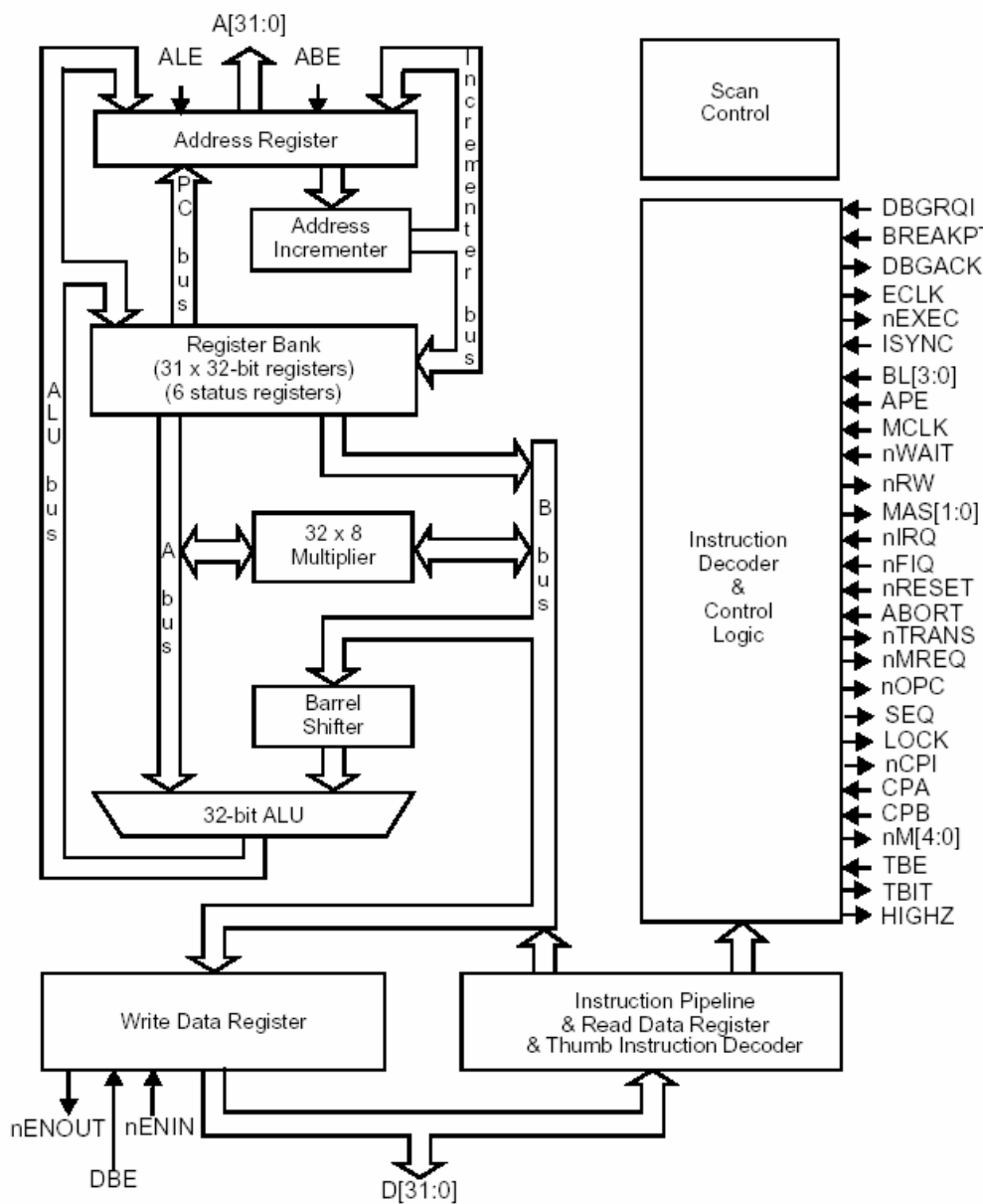
- หน่วยประมวลผลมีชุดคำสั่ง 2 แบบ ได้แก่ ชุดคำสั่ง ARM 32 บิต และชุดคำสั่ง Thumb ขนาด 16 บิต โดย คำสั่งเหล่านี้สามารถที่ทำการประมวลผลกับข้อมูลได้หลายขนาด เช่น 8 , 16 , 32 บิต
- หน่วยประมวลผลมีโหมดการทำงาน 7 โหมด โดยแต่ละโหมดมีรีจิสเตอร์เพื่อเพิ่มความเร็วในการจัดการแอ็กเซ็ปชั่น (Exception)
- โปรเซสเซอร์มีรีจิสเตอร์ขนาด 32 จำนวน 37 ตัว รวมถึง 6 รีจิสเตอร์สถานะ

ARM7TDMI Block Diagram



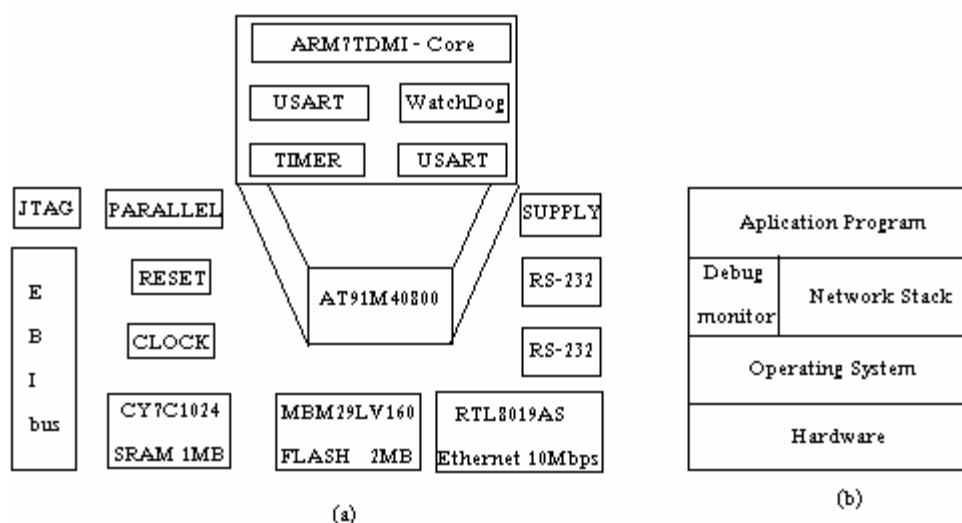
รูปที่ 3.2 โครงสร้างของ ARM7TDMI

ARM7TDMI Core Diagram



รูปที่ 3.3 โครงสร้างภายในของ ARM7TDM

3.3.3 สถาปัตยกรรม



รูปที่ 3.4 (a) โครงสร้าง (b) สถาปัตยกรรมทางด้านซอฟต์แวร์

3.3.4 ไมโครคอนโทรลเลอร์ AT91M40800

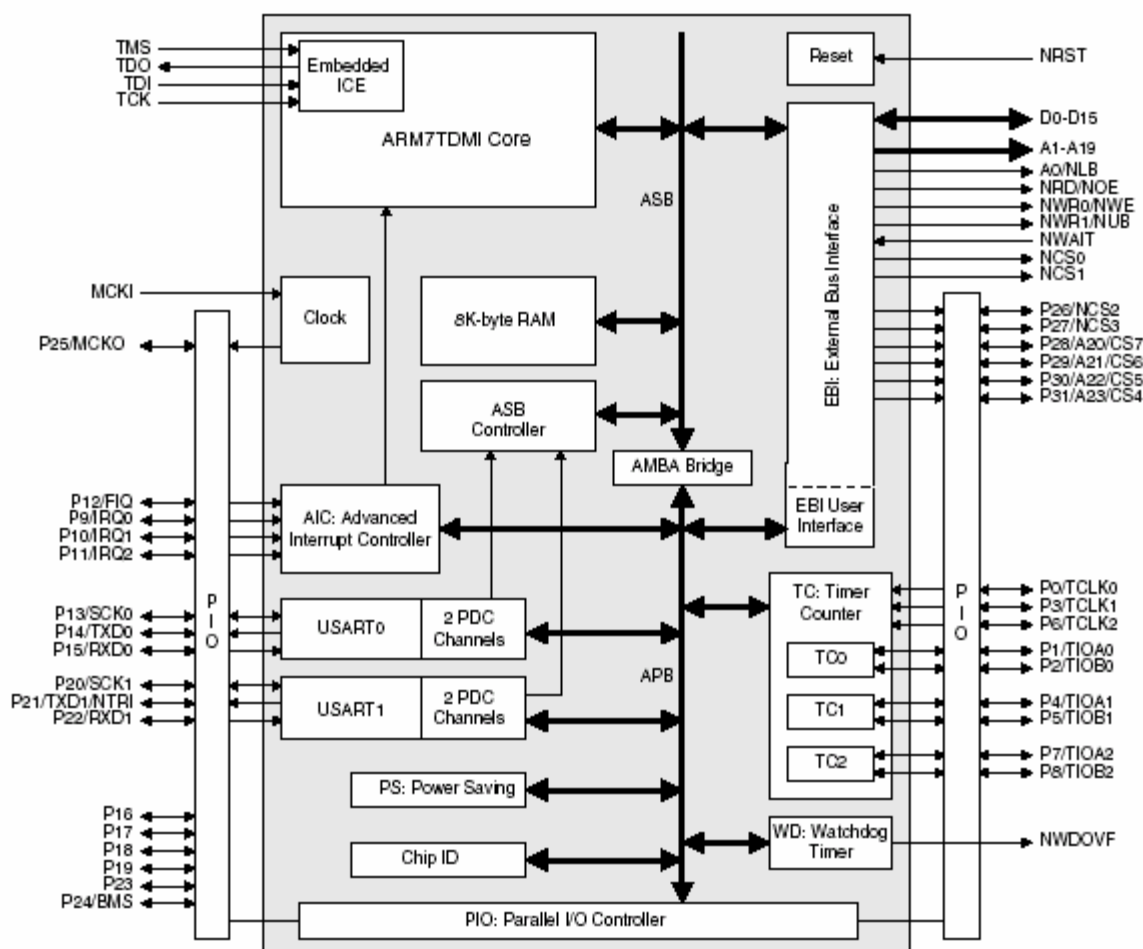
ไมโครโปรเซสเซอร์ที่เลือกใช้ในการออกแบบบอร์ดควบคุม AT91M40800 เป็นไมโครคอนโทรลเลอร์ ARM7TDMI 16/32 บิต สถาปัตยกรรม RISC ความเร็ว 30 MIPS ที่สัญญาณคล็อก 33 MHz ขณะที่สิ้นเปลืองกำลังงานน้อยกว่าหนึ่งวัตต์ ภายในได้รวมเอาอุปกรณ์ที่เป็นพื้นฐานในการพัฒนาระบบฝังตัว ได้แก่

- 8 Kbyte SRAM 32 บิตสามารถอ่านและเขียนใน single cycle clock
- อินเทอร์รัพท์คอนโทรลเลอร์ 8-level Priority, Individually Maskable, Vectored Interrupt Controller และอินเทอร์รัพท์จากภายนอก 4 เส้น
- พอร์ต Parallel I/O สามารถโปรแกรมได้ 32 เส้น
- 2 USART (Universal Asynchronous Receiver/Transmitter) สำหรับการสื่อสารอนุกรมความเร็วสูง
- Programmable Timer/Counter 16 บิต 3 ตัว
- Programmable Watchdog Timer
- Power Saving Controller

สถาปัตยกรรมของ AT91M40800

ประกอบด้วยบัสหลัก 2 เส้นซึ่งเชื่อมต่อกันด้วยโดย AMBA Bridge คือ

ASB (The Advanced System Bus) ถูกออกแบบสำหรับประสิทธิภาพที่สูงที่สุด ทำการเชื่อมต่อ



รูปที่ 3.5 บล็อกไดอะแกรมของไมโครคอนโทรลเลอร์ AT91M40800

โปรเซสเซอร์ หน่วยความจำบนชิพขนาด 32 บิตและหน่วยความจำภายนอกรวมทั้งอุปกรณ์ต่างๆผ่าน EBI (External Bus Interface)

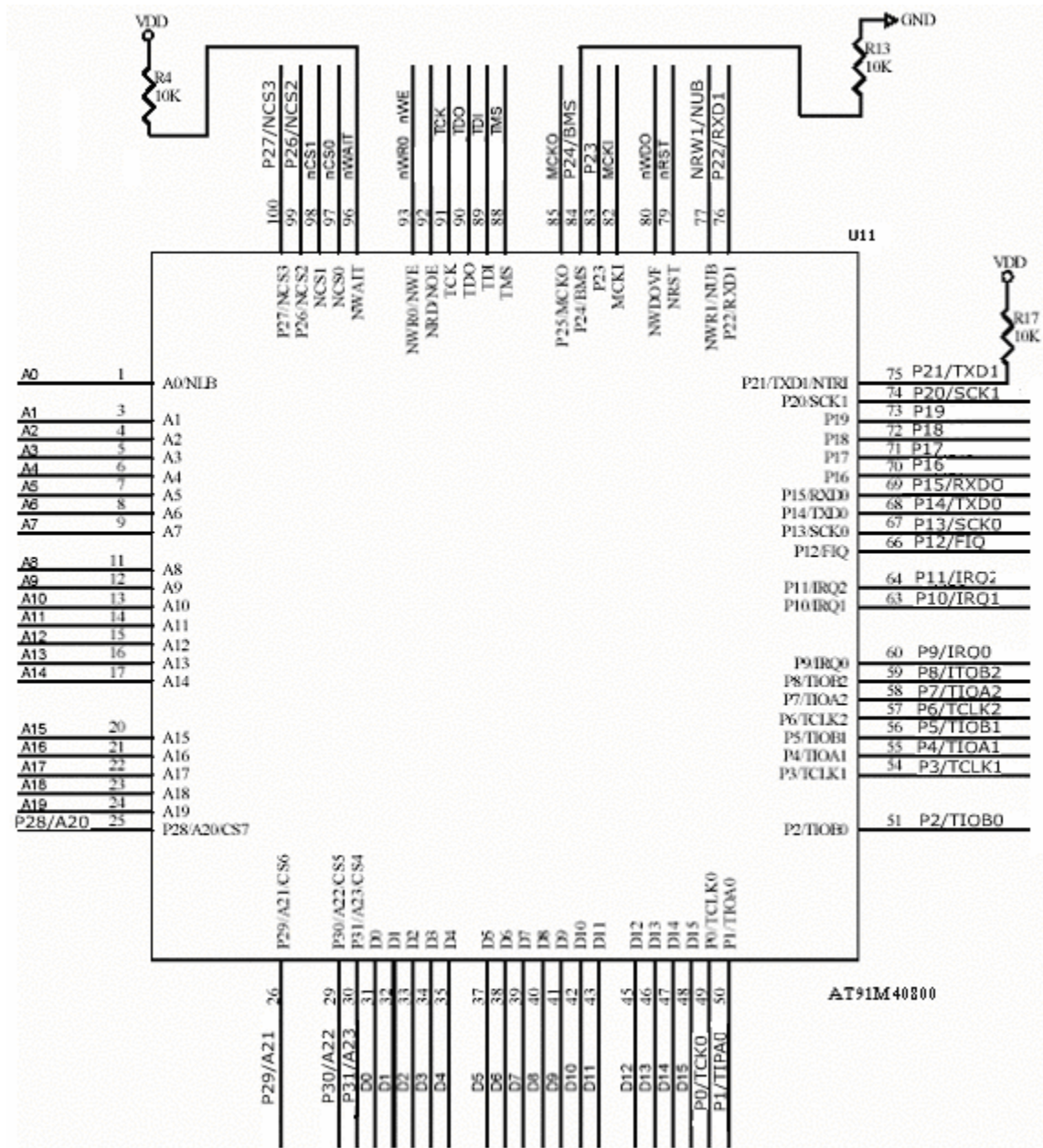
APB (The Advanced Peripheral Bus) ถูกออกแบบสำหรับเข้าถึงอุปกรณ์ต่างๆบนชิพและใช้พลังงานน้อย

PDC (Peripheral Data Controller) ทำการโอนย้ายข้อมูลระหว่าง USARTs ที่อยู่บนชิพกับหน่วยความจำทั้งภายในและนอกชิพที่สำคัญ PDCจะทำการลดโอเวอร์ฮีดที่โปรเซสเซอร์ทำการอินเตอร์ แชนดิ่ง และลดจำนวนไซเคิลของสัญญาณนาฬิกา ที่ต้องการสำหรับโอนย้ายข้อมูล ซึ่งสามารถทำการโอนย้ายขนาด 64 กิโลไบต์ได้อย่างต่อเนื่อง ทำให้เพิ่มประสิทธิภาพและลดการสิ้นเปลืองพลังงานลงได้

อุปกรณ์รอบข้างภายในถูกออกแบบให้ใช้ชุดคำสั่งในการทำงานน้อยในการทำงานแต่โดยแต่ละตัวจะมีช่วงแอดเดรสขนาด 16 กิโลไบต์ในขอบเขต 3 เมกกะไบต์บนช่วงของ 4 แอดเดรสกิกะไบต์ เพื่อให้ประสิทธิภาพ

ภาพในการดำเนินงานกับบิตยังได้มีการแยกจิสเตอร์ ที่มีการใช้บ่อยเป็น 3 ตำแหน่งและให้ตำแหน่งแรก เป็นการเซตบิต ตำแหน่งที่สอง เป็นการเคลียร์บิต และตำแหน่งที่สามเป็นการอ่านค่าของรีจิสเตอร์นั้นๆ

สัญญาณภายนอกทั้งหมดของอุปกรณ์รอบข้างจะถูกควบคุมโดย PIO (Parallel I/O Controller) ซึ่งสามารถโปรแกรมให้ ตัวมันเองเพิ่มตัวกรองอินพุตได้



รูปที่ 3.6 ไมโครโปรเซสเซอร์ Schematic diagram

3.3.5 หน่วยความจำระบบ

บอร์ดควบคุมที่สร้างขึ้นมีหน่วยความจำระบบดังนี้

1. MBM29LV160 เป็นหน่วยความจำแฟลชขนาด 2 เมกกะไบต์ มีความสามารถโปรแกรมได้ไม่น้อยกว่า 100,000 ครั้ง การเข้าถึงข้อมูล 90 นาโนวินาที(ns) สำหรับเก็บการทำงานเริ่มต้นของระบบ
2. SRAM เบอร์ CYC1049 ขนาด 512 เมกกะไบต์ 2 ตัว เวลาในการเข้าถึง 15 นาโนวินาที ทำให้ไม่ต้องใช้ wait state

3.3.6 การจัดการหน่วยความจำ

ไมโครคอนโทรลเลอร์ AT91M40800 สามารถโปรแกรมแต่เลือกชิพได้ 8 ตัว หรือ 4 ตัว แต่ในบอร์ดควบคุมที่ได้ออกแบบไว้ให้เลือกการทำงานได้กับชิพ 4 ตัว แต่ละตัวสามารถอ้างหน่วยความจำได้ 16 MB ทำให้สามารถอ้างหน่วยความจำได้ทั้งหมด 64 MB สัญญาณที่ใช้ในการเลือกชิพคือ NCS0 ถึง NCS3 สามารถที่จะโปรแกรมเลือกขนาด Data bus ได้ทั้ง 8 หรือ 16 บิต ตารางข้างล่างแสดงการทำ Memory Map

ตารางที่ 3.3 การจัดการหน่วยความจำ (Memory Map)

Address Range	Function	Chip Select
0x00000000 – 0x00000200	Internal SRAM 8 Kbyte	-
0x01000000 - 0x011FFFFFFF	Flash Memory 2Mbyte	NCS0
0x02000000 – 0x020FFFFFFF	External SRAM 1 Mbyte	NCS1
0x03000000 - 0x03000400	Ethernet Controller	NCS2
0xFFC00000 – 0xFFFFFFFF	On-Chip peripherals	-

3.3.7 หน่วยความจำ I/O

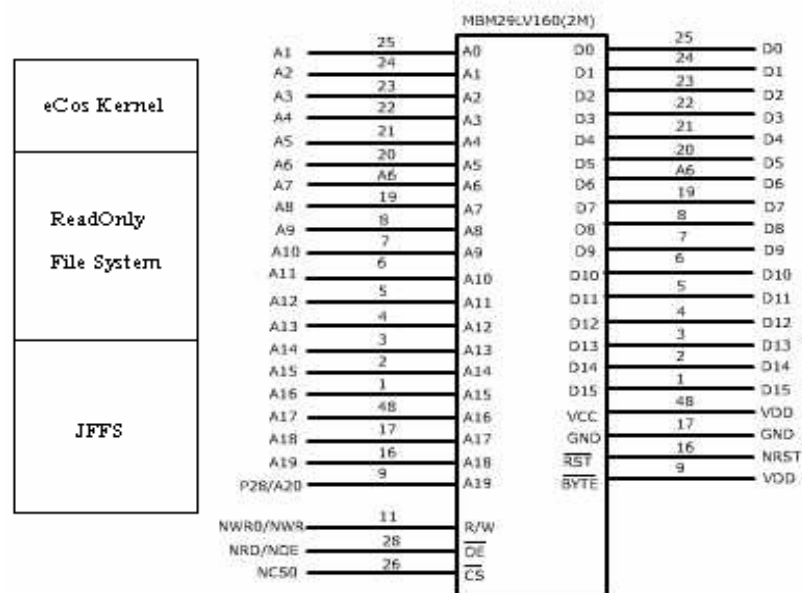
หน่วยความจำ I/O สามารถแบ่งได้เป็น 2 ส่วนในแอดเดรสทั้งหมดของ ARM7TDMI 4 GB คือ พื้นที่แอดเดรสของ EBI-bus (Externale Bus Interface) ตั้งแต่ 0x00400000 – 0xFFBFFFFFFF ซึ่งสามารถเลือกโดย NCS3 ซึ่งไม่ถูกใช้งานในขณะนี้ และส่วนที่สองเป็น On-Chip Peripherals ตั้งแต่แอดเดรส 0xFFC00000 – 0xFFFFFFFF แต่ไม่ได้ถูกใช้งานทั้งหมด การจัดหน่วยความจำของ I/O อ้างถึงใน DATA Sheet ของ AT91x4x ของ Atmel ARM7TDMI ไมโครคอนโทรลเลอร์ ดังนี้

ตารางที่ 3.4 การจัดการหน่วยความจำ I/O (Memory Map I/O)

Address Range	Function	Chip Select
0xFFE00000 – 0xFFE03FFF	EBI-external bus interface 16 kb	-
0xFFFF0000 – 0xFFFF03FFF	Special Function 16 Kbyte	-
0xFFFFCC00 - 0xFFFFC3FFF	USART1 16 Kbyte	-
0xFFFFD0000 – 0xFFFFD3FFF	USART0 16 KByte	-
0xFFFE0000 – 0xFFFE3FFF	Timer Counter 16 KByte	-
0xFFFF0000 – 0xFFFF3FFF	PIO 16 Kbyte	-
0xFFFF4000 – 0xFFFF7FFF	Power saving	-
0xFFFF8000 – 0xFFFFBFFF	Watchdog Timer	-
0xFFFFF000 – 0xFFFFFFF	Advance interrupt Controller	-

3.3.8 หน่วยความจำแฟลช (Flash Memory)

หน่วยความจำแฟลช(Flash ROM) ที่ใช้งานกับบอร์ดความคุมคือ ฟุจิชิ (Fujishu) เบอร์ MBM29LV160 อ่านและเขียนด้วยไฟเลี้ยง 3.0V มีขนาดของ 2 Mbyte จะถูกกำหนดไว้ที่แอดเดรส 0x01000000 หลังจากทำการรีแม็บ (Remap memory) รูปที่ 3.7 เป็นการแสดงวงจรที่ต่อใช้งาน



รูปที่ 3.7 แฟลช Schematics Diagram และ Memory Layout

Layout ของหน่วยความจำแฟลช

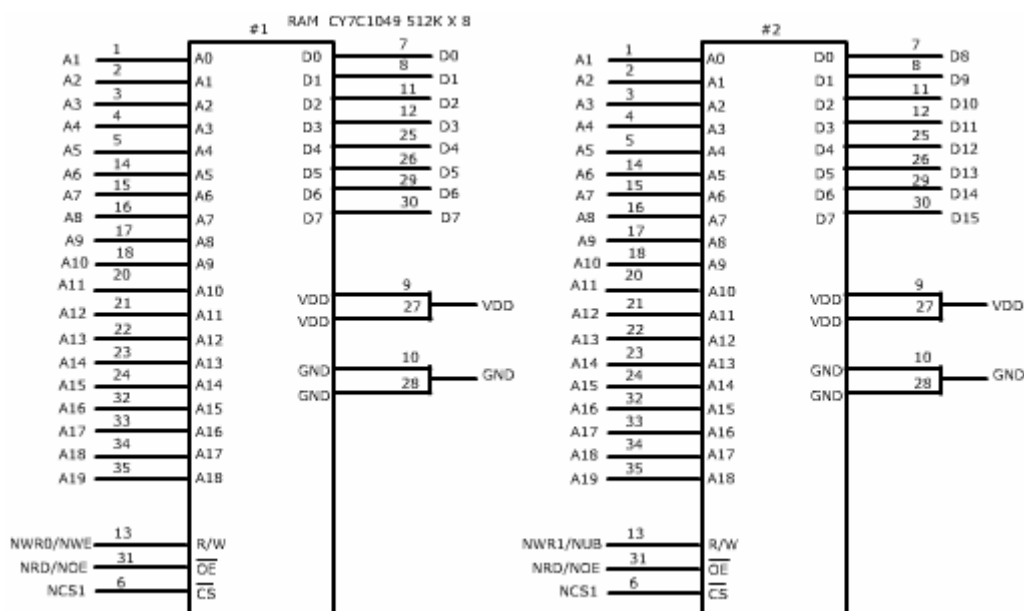
สำหรับบอร์ดควบคุมที่ได้ออกแบบไว้จะเก็บข้อมูลสำหรับระบบดังนี้

- eCos Kernel ที่เริ่มต้นของหน่วยความจำแฟลช
- ไฟล์ระบบจะถูกเรียกใช้โดย eCos Kernel สามารถอ่านได้อย่างเดียว
- JFFS(Journaling Flash File System) ใช้ในการเข้าใช้งานของแอปพลิเคชัน

สิ่งที่ต้องพิจารณาเป็นกรณีพิเศษคือ หน่วยความจำแฟลชจะถูกอ่านในครั้งแรกเมื่อจ่ายไฟให้กับบอร์ดควบคุมหรือทำการรีเซ็ต ซึ่งถูกเรียกใช้งานโดย NCS0 อยู่ที่แอดเดรส 0x00000000 ดังนั้นตำแหน่งเริ่มต้นของแฟลชจะเก็บโค้ดโปรแกรมบูทระบบ หลังจากทำการรีเซ็ตแล้ว หน่วยความจำแฟลช จึงจะถูกกำหนดที่แอดเดรส 0x01000000 และหน่วยความจำที่ตำแหน่ง 0x00000000 จะกลายเป็น SRAM ภายใน 8Kbyte

3.3.9 หน่วยความจำแรม

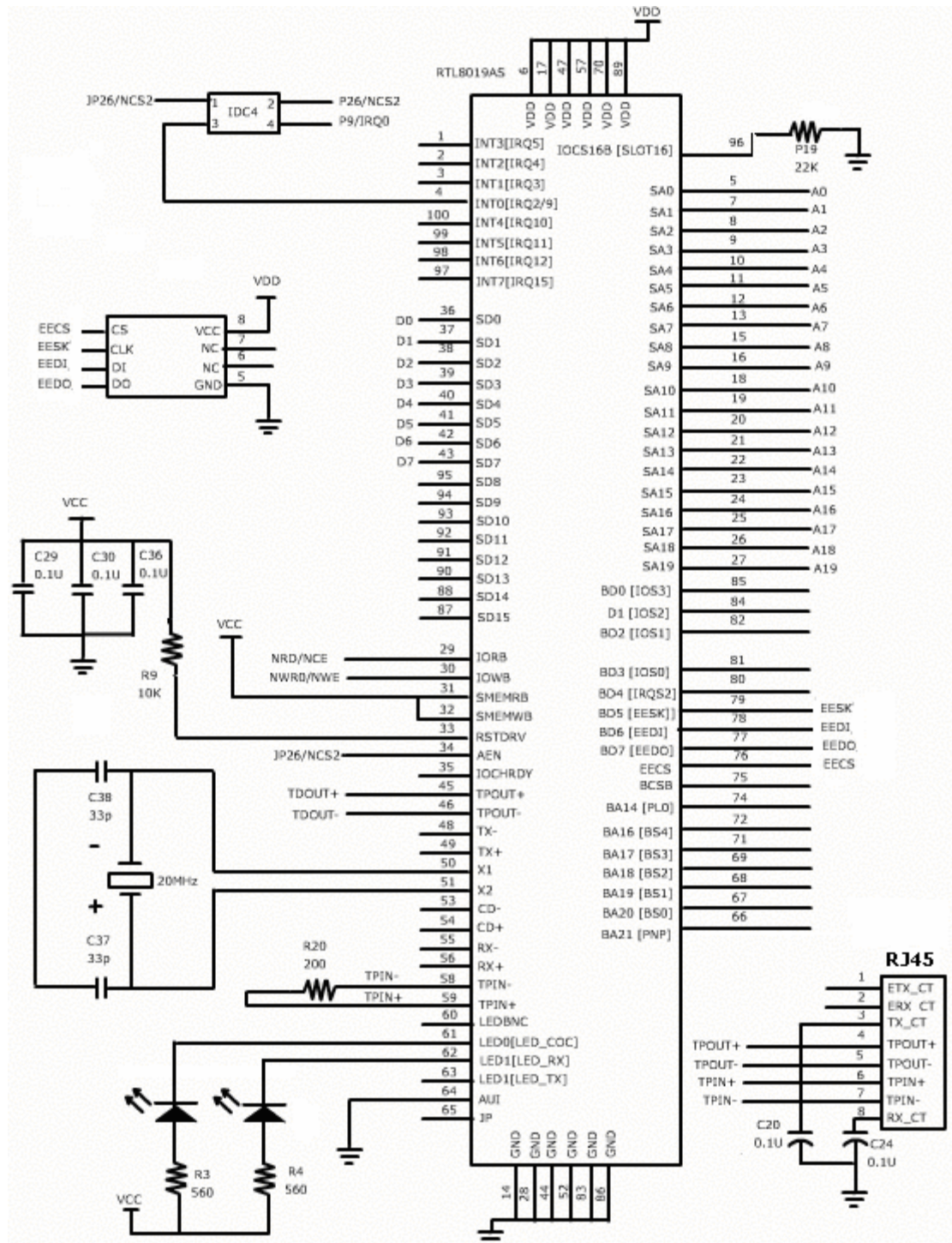
แรมที่ใช้ในบอร์ดควบคุมคือ CY7C1024 ขนาด 512K x 8 ทั้งหมด 2 ตัวรวมเป็น 1 MB ใช้ไฟเลี้ยง 3.0V สามารถทำงานร่วมกับไมโครคอนโทรลเลอร์โดยไม่มี Wait State ซึ่งจะถูกกำหนดไว้ที่ แอดเดรส 0x02000000 จะเรียกใช้ในขณะรันโปรแกรม รูปที่ แสดงการ Schematic ของ SRAM



รูปที่ 3.8 Schematic diagram ของ SRAM

3.3.10 Ethernet Controller

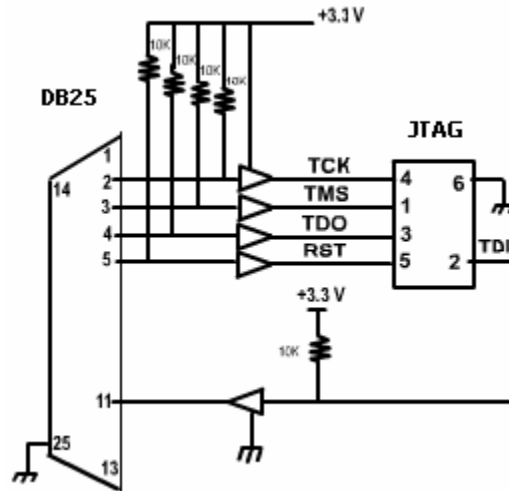
ในบอร์ดควบคุมได้ทำการเพิ่มความสามารถในการติดต่อผ่านเครือข่ายคอมพิวเตอร์ โดยติดตั้ง Ethernet Controller เบอร์ RealTek RTL8019AS ความเร็ว 10 Mbps ต่อเข้า data bus 8 bit ถูกกำหนดในพื้นที่ I/O แอดเดรส 0x03000000 โดยใช้งานสัญญาณเลือกชิพ NCS2 รูปที่ ค แสดงวงจรที่ใช้ในการออกแบบ



รูปที่ 3.9 Schematics Diagram ของภาค Ethernet Controller

3.3.11 พอร์ต JTAG

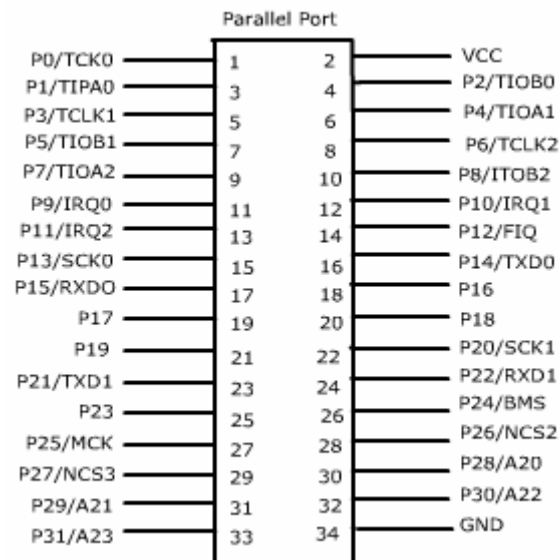
สำหรับในระบบฝังตัวแล้วอุปกรณ์ทุกอย่างจะยึดเข้ากับแผงวงจร เพื่อให้มีขนาดเล็ก และราคาถูก ดังนั้นเมื่อต้องการที่จะตรวจสอบข้อผิดพลาด หรือแม้กระทั่งการปรับปรุงซอฟต์แวร์ต้องมีอินเตอร์เฟสสำหรับติดต่อกับอุปกรณ์ JTAG(Joint Test Access Group) รูปที่ 3.10 แสดงการต่อพอร์ต JTAG



รูปที่ 3.10 การเชื่อมต่อ JTAG พอร์ต

3.3.12 พอร์ตขนาน (PIO)

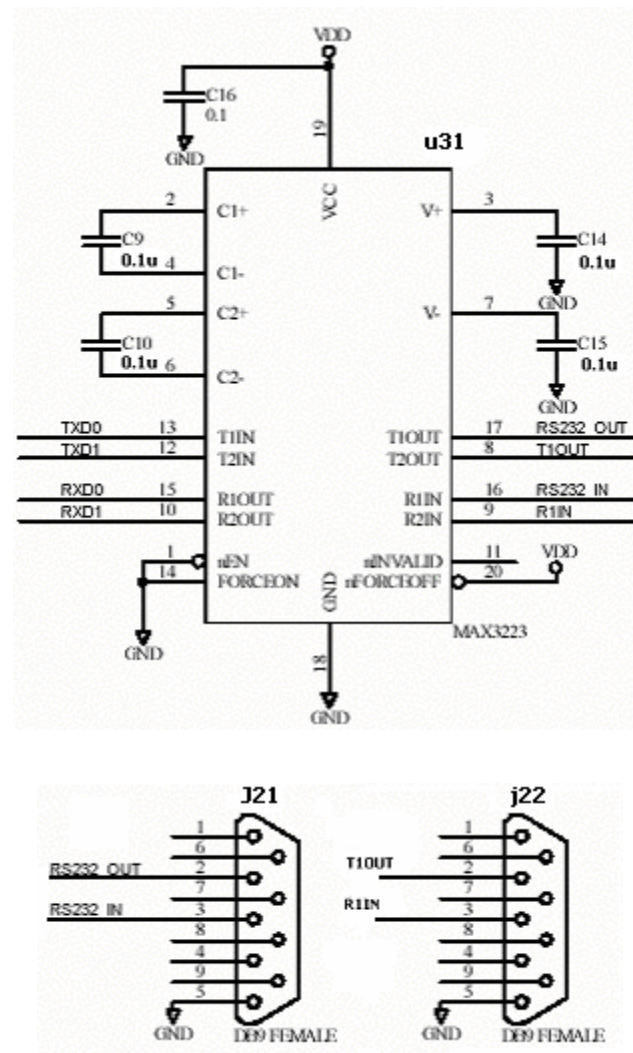
พอร์ตขนานเป็นอีก I/O พอร์ตที่มีอยู่ในบอร์ดควบคุมมีขนาด 32 เส้น ซึ่งสามารถที่จะโปรแกรมใช้งานได้ทั้งหมด สำหรับแอดเดรสที่ใช้การอ้าง PIO อยู่ที่ 0xFFFF0000 – 0xFFFF3FFF



รูปที่ 3.11 สัญญาณที่เชื่อมต่อกับ PIO

3.3.13 USART

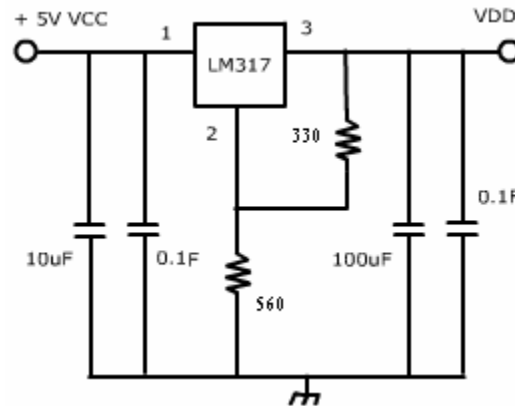
ในบอร์ดควบคุมที่ได้ออกแบบขึ้นที่อินเทอร์เฟซกับ USART (Universal Asynchronous Receiver/Transmitter) สำหรับการสื่อสารแบบอนุกรม ซึ่งจะถูกต้องเข้ากับ DB9-RS232 สามารถรองรับการสื่อสารที่ความเร็วสูงสุด 230400 bps เป็นการเชื่อมต่อแบบ NULL Modem และ แอดเดรสของ USART ทั้งสอง 2 ช่องจะอยู่ที่ 0xFFFD0000 – 0xFFFD3FFF สำหรับ USART0 และ 0xFFFC000 - 0xFFFC3FFF สำหรับ USART1 ในการติดต่อกับโฮส(Host System) จะกำหนดไว้ที่ USART0 รูปที่ 3.12 เป็นวงจรที่ใช้งานการติดต่อสื่อสารอนุกรม โดยมีไดรเวอร์เป็นไอซี เบอร์ MAX3223



รูปที่ 3.12 USART Schematic Diagram

3.3.1.4 ภาคจ่ายไฟ(Power Supply)

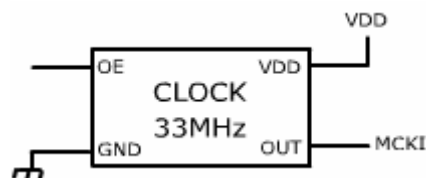
เนื่องจากไฟที่เลี้ยงวงจรส่วนใหญ่ใช้แรงดัน 3.3 โวลต์ แต่ไฟที่จ่ายให้กับบอร์ดควบคุมนั้นมีค่า 5 โวลต์ ดังนั้นต้องทำการควบคุมแรงดันให้เหลือเพียง 3.3 โวลต์ IC ที่ใช้คือ LM317 รูปที่ 3.12 . แสดงวงจรของภาคจ่ายไฟที่ใช้งานจริง



รูปที่ 3.13 วงจรภาคจ่ายไฟ

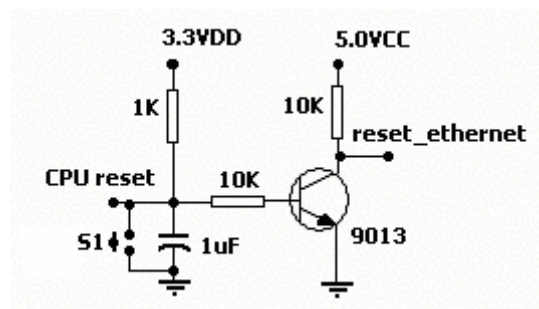
3.3.1.5 ภาคกำเนิดสัญญาณคล็อก(Clock Generator)

ทำหน้าที่กำเนิดสัญญาณคล็อกให้กับไมโครคอนโทรลเลอร์ ความถี่ 33 Mhz รูปข้างล่างแสดงการต่อใช้งาน



รูปที่ 3.14 Schematic Diagram ของภาคกำเนิดสัญญาณคล็อก

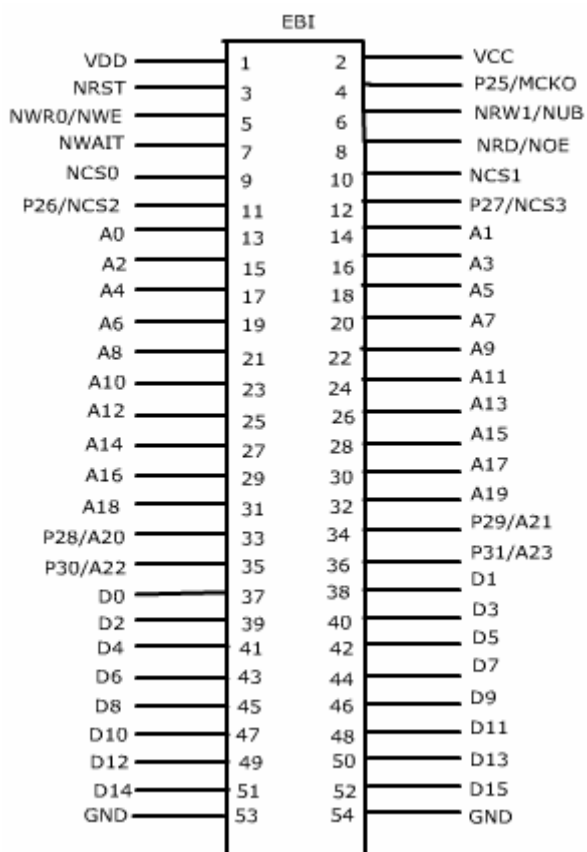
3.3.1.6 ภาคสัญญาณรีเซ็ต (Reset signal)



รูปที่ 3.14 Schematic Diagram ภาคสัญญาณรีเซ็ต

3.3.17 พอร์ต EBI(External Bus Interface)

EBI Ports จะถูกใช้ในการขยายหน่วยความจำและอุปกรณ์ I / O จากบอร์ดควบคุมรูปที่ 11 พอร์ต EBI ซึ่งจะรวมเอาสัญญาณควบคุม ,แอดเดรสบัส และ คาต้าบัส ไว้ จะมีทั้งหมด 54 ขา



รูปที่ 3.16 การต่อวงจรเข้ากับ EBI-บัส

3.3.18 คุณสมบัติทางไฟฟ้า

- ทำงานได้กับแหล่งจ่าย 5 VDC เท่านั้น
- กินกระแสสูงสุด 750 mA@5VDC

3.3.19 อุณหภูมิทำงาน

- อุณหภูมิทำงาน 0 ถึง +70 องศาเซลเซียส

3.4 ส่วนประกอบทางด้านซอฟต์แวร์

ในส่วนนี้ได้ทำการวิเคราะห์เพื่อหาซอฟต์แวร์ที่ใช้ในการพัฒนาที่สะดวกและเหมาะสม โดยสามารถที่จะทำการพัฒนาได้บนเครื่องคอมพิวเตอร์ทั่วไปได้ รวมทั้งมีเครื่องมือให้ได้ง่าย โดยตัวเครื่องที่ใช้พัฒนาจะต้องทำงานด้วยระบบปฏิบัติการลินุกซ์ และมีการใช้คอมไพเลอร์ที่สามารถที่จะแปลงไฟล์จากระบบ x86 ไปเป็น ARM ได้ รวมทั้งมีเครื่องมือในการโหลดข้อมูลลงไปทำงานบนบอร์ดผ่านทางพอร์ตขนานของเครื่องพีซีได้

3.4.1 ระบบปฏิบัติการที่น่าสนใจสำหรับระบบฝังตัว

3.4.1.1 uClinux

พัฒนามาจากลินุกซ์เคอร์เนล 2.0 มีจุดประสงค์เพื่อใช้ในงานไมโครคอนโทรลเลอร์โดยไม่ต้องอาศัย การจัดการของหน่วยความจำ (MMUs) มีความสามารถในการทำงานแบบ multitasking นอกจากนั้นไฟล์ไบนารี และซอร์สโค้ดสำหรับเคอร์เนล ยังถูกเขียนใหม่ให้มีขนาดเล็กลงและลดจำนวนโค้ดที่ใช้ให้น้อยที่สุด มีความเสถียร สนับสนุนการทำงานด้านเน็ตเวิร์ก และไฟล์ระบบ มีขนาดของเคอร์เนล ไม่เกิน 512 กิโลไบต์ และเมื่อรวมส่วนประกอบที่เป็นเครื่องมืออื่นๆ ไปด้วยแล้วจะไม่เกิน 900 กิโลไบต์

คุณสมบัติของ uClinux

- MMU non-dependent
- สนับสนุนโปรโตคอล TCP/IP, SLIP และ PPP
- Embedded Web servers
- Embedded uC library collection uClibc, uClibns, uCcrypt
- ไฟล์ระบบที่สนับสนุน : NFS / JFFS / bFLAT EXT2FS / MINIX / SMB
- fully customizable

แพลตฟอร์มที่รองรับ

- Motorola
- Atmel
- Samsung
- Conexant
- Analog Devices
- Hitachi
- SPARC
- Intel
- MIPS 4K
- และอื่น ๆ

ขนาดของโค้ด

- เคอร์เนลโดยปกติมีขนาดประมาณ 512 กิโลไบต์
- เมื่อรวมองค์ประกอบทั้งหมดจะมีขนาดประมาณ 1 เมกะไบต์

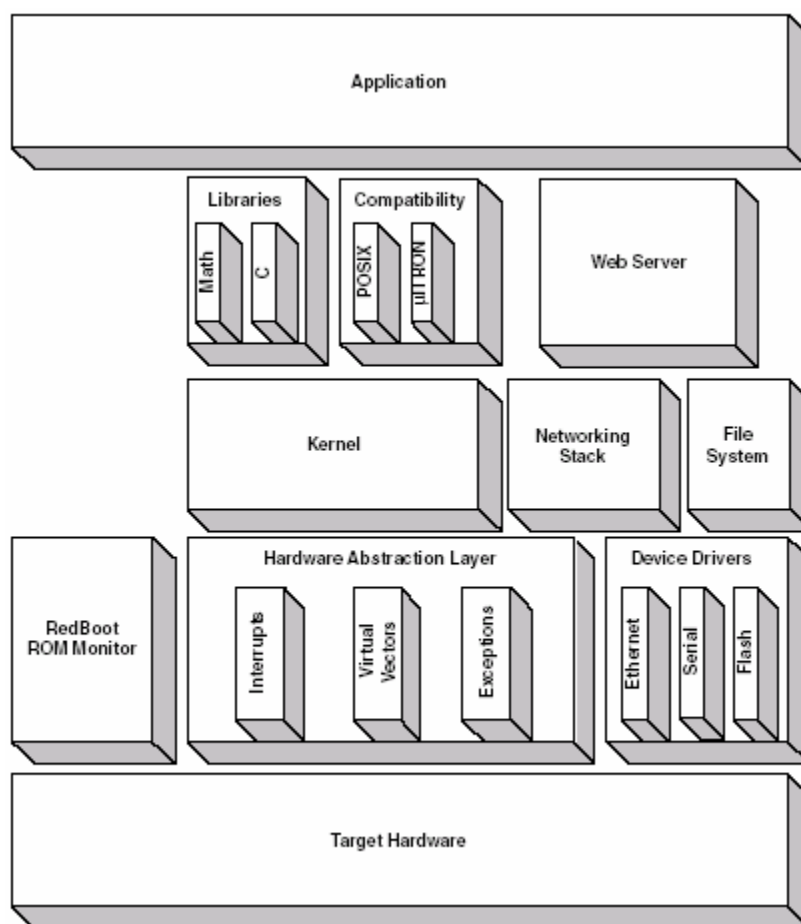
3.4.1.2 eCos (Embedded Configurable Operating System)

eCos เป็นระบบปฏิบัติการพัฒนาเป็นครั้งแรกในปี 1997 เป้าคือเพื่อพัฒนาระบบฝังตัวประสิทธิภาพสูง ต้องการทรัพยากรที่น้อยลง นอกจากนี้ยังเป็นระบบปฏิบัติการที่มีการเผยแพร่แจกจ่ายซอร์สโค้ดฟรี ถูกออกแบบให้สามารถรองรับแอปพลิเคชันต่างๆ ตามที่ผู้พัฒนาต้องการต้องการไม่ว่าจะเป็นด้านการติดต่อสื่อสาร การควบคุมเครื่องจักร และแอปพลิเคชันอื่นๆ ในระดับสูง

eCos เป็นระบบปฏิบัติการที่เป็นเรียลไทม์ (Real – Time Operating System) มีความเหมาะสมกับระบบฝังตัวระดับลึกที่ระบบปฏิบัติการลินุกซ์ทั่วไปไม่สามารถทำได้ โดยทั่วไประบบปฏิบัติการลินุกซ์ จะมีขนาดของเคอร์เนล อยู่ที่ประมาณ 500 กิโลไบต์ ถึง 1.5 เมกะไบต์ แต่ eCos จะมีขนาดของเคอร์เนลประมาณ 10 - 100 กิโลไบต์ และอาจขึ้นอยู่กับความต้องการของระบบด้วย

สถาปัตยกรรม

eCos ถูกออกแบบมาให้สนับสนุนกับแอปพลิเคชันที่มีการทำงานแบบเรียลไทม์(real-time) เหมาะที่จะนำมาพัฒนาแอปพลิเคชันของระบบฝังตัว ที่ระบบปฏิบัติการลินุกซ์ทั่วไปไม่สามารถทำได้ โดยทั่วไประบบปฏิบัติการลินุกซ์ จะมีขนาดของเคอร์เนล อยู่ที่ประมาณ 500 กิโลไบต์ ถึง 1.5 เมกะไบต์ แต่ eCos จะมีขนาดของเคอร์เนลประมาณ 10 - 100 กิโลไบต์ และอาจขึ้นอยู่กับความต้องการของระบบด้วย รองรับกับไดรเวอร์ของอุปกรณ์ ,ไฟล์ระบบ ,โพรโตคอล TCP/IP การจัดการหน่วยความจำ , exception handling , ภาษาซีและไลบรารี , และอื่น ๆ eCos มีเครื่องมือที่ใช้ในการ



รูปที่ 3.17 สถาปัตยกรรมของระบบปฏิบัติการ eCos

พัฒนาแอปพลิเคชันระบบฝังตัว โดยได้รับการสนับสนุนตัวคอมไพเลอร์, แอสเซมเบลอร์, ลิงเกอร์, ดีบั๊กเกอร์ และซิมูเลเตอร์ จาก กนู(GNU) ดังนี้

- สนับสนุนไครว์เวอร์อีเทอร์เน็ต USB , flash , serial และอื่นๆ
- สนับสนุนการทำงาน TCP/IP แบบ SNMP
- สนับสนุนภาษาซีและไลบรารี
- รองรับมาตรฐาน POSIX EL/XI ระดับ 1
- สนับสนุน μITRON 3.02
- ได้รับการสนับสนุนเครื่องมือจากกนู(GNU)
- สนับสนุนดีบั๊กมอนิเตอร์ RedBoot
- มี Hardware Abstraction Layer (HAL) ทำให้สามารถทำงานได้กับโปรเซสเซอร์หลายแพลตฟอร์ม
- ส่วนประกอบของเคอร์เนลที่เป็นเรียลไทม์ :

- Interrupt handling
- Exception handling
- Choice of schedulers
- Thread support
- Rich set of synchronization primitives
- Timers, counters, and alarms
- Choice of memory allocators
- Debug and instrumentation support

สนับสนุนระบบปฏิบัติการโฮสต์

- Red Hat Linux
- Sun Solaris and ระบบ UNIX อื่นๆ
- Microsoft Windows

แพลตฟอร์มที่สนับสนุน

- ARM
- Hitachi SH3, SH4
- MIPS
- NEC V850
- Panasonic AM3x
- PowerPC
- SPARC
- x86
- SuperH, Matsushita AM3x, IA32, และอื่นๆ

3.4.2 เหตุผลที่เลือกใช้ระบบปฏิบัติการ eCos

จากที่ได้ลองศึกษาระบบปฏิบัติการที่ผ่านมาของโครงการนี้ มีระบบปฏิบัติการอยู่ 2 ตัว ที่นำมาพิจารณาเลือกใช้ คือ eCos และ uClinux โดยระบบปฏิบัติการทั้ง 2 ตัว นี้มีคุณสมบัติที่แตกต่างกันอยู่ ดังนั้นการพิจารณาเลือกระบบปฏิบัติการที่จะนำมาใช้ ต้องมีความเหมาะสมและเข้ากันได้กับโครงการนี้

จากที่ได้ลองพิจารณา พบว่าระบบปฏิบัติการ eCos มีข้อได้เปรียบกว่า uClinux อยู่หลายด้าน เนื่องจาก eCos เป็นระบบปฏิบัติการที่พัฒนามาเพื่อระบบฝังตัวโดยเฉพาะ มีคุณสมบัติที่ดีกว่าเหมาะที่จะนำมาใช้ในการพัฒนาระบบฝังตัว และดีกว่า uClinux ที่ใช้กันอยู่ในปัจจุบันนี้ uClinux พัฒนามาจากลินุกซ์และ ทำให้มีคุณสมบัติที่เหมาะสมน้อยกว่า เมื่อเทียบกับ eCos เนื่องจาก uClinux ไม่ได้พัฒนามาใช้สำหรับระบบฝังตัวโดยเฉพาะ

ตารางที่ 3.5 แสดงการเปรียบเทียบระหว่างระบบปฏิบัติการ eCos และ uClinux

ข้อพิจารณา	eCos	uClinux
แจกจ่ายฟรี เผยแพร่ซอร์สโค้ด	ใช่	ใช่
ทำงานบนไมโครโพรเซสเซอร์แบบ 32 บิต	ใช่	ใช่
สนับสนุนการทำงานด้านเน็ตเวิร์ก	ใช่	ใช่
แหล่งข้อมูล	มีมาก	มีน้อย
การทำงานแบบ Real - time	ใช่	ไม่ใช่
ขนาดของเคอร์เนล (กิโลไบต์)	10 - 100	600

จากตารางจะเห็นว่า ขนาดของเคอร์เนลที่พัฒนามาจาก uClinux มีขนาดอย่างน้อยประมาณ 600 กิโลไบต์ ซึ่งไม่ เหมาะสมที่นำมาใช้กับระบบฝังตัว แต่ eCos จะมีขนาดแค่เพียง 10 - 100 กิโลไบต์เท่านั้น โดยจะขึ้นอยู่กับความต้องการของระบบ นอกจากนั้น uClinux ยังไม่มีคุณสมบัติทางด้าน “Real – time” ถึงแม้ว่าจะนำมาใช้งานแบบเดียวกัน แต่สำหรับ eCos จะมีคุณสมบัติการทำงานแบบ “Real – time” พร้อมอยู่ในตัวแล้ว นอกจากนั้น eCos มีเครื่องมือที่ใช้ในการพัฒนาระบบฝังตัวที่ดีกว่าของ uClinux

จากที่ได้ลองพิจารณาเปรียบเทียบระบบปฏิบัติการ eCos และ uClinux จึงทำให้ได้ข้อสรุปว่า ระบบปฏิบัติการ eCos น่าสนใจที่จะนำมาใช้ในการพัฒนากับโครงการนี้มากที่สุด

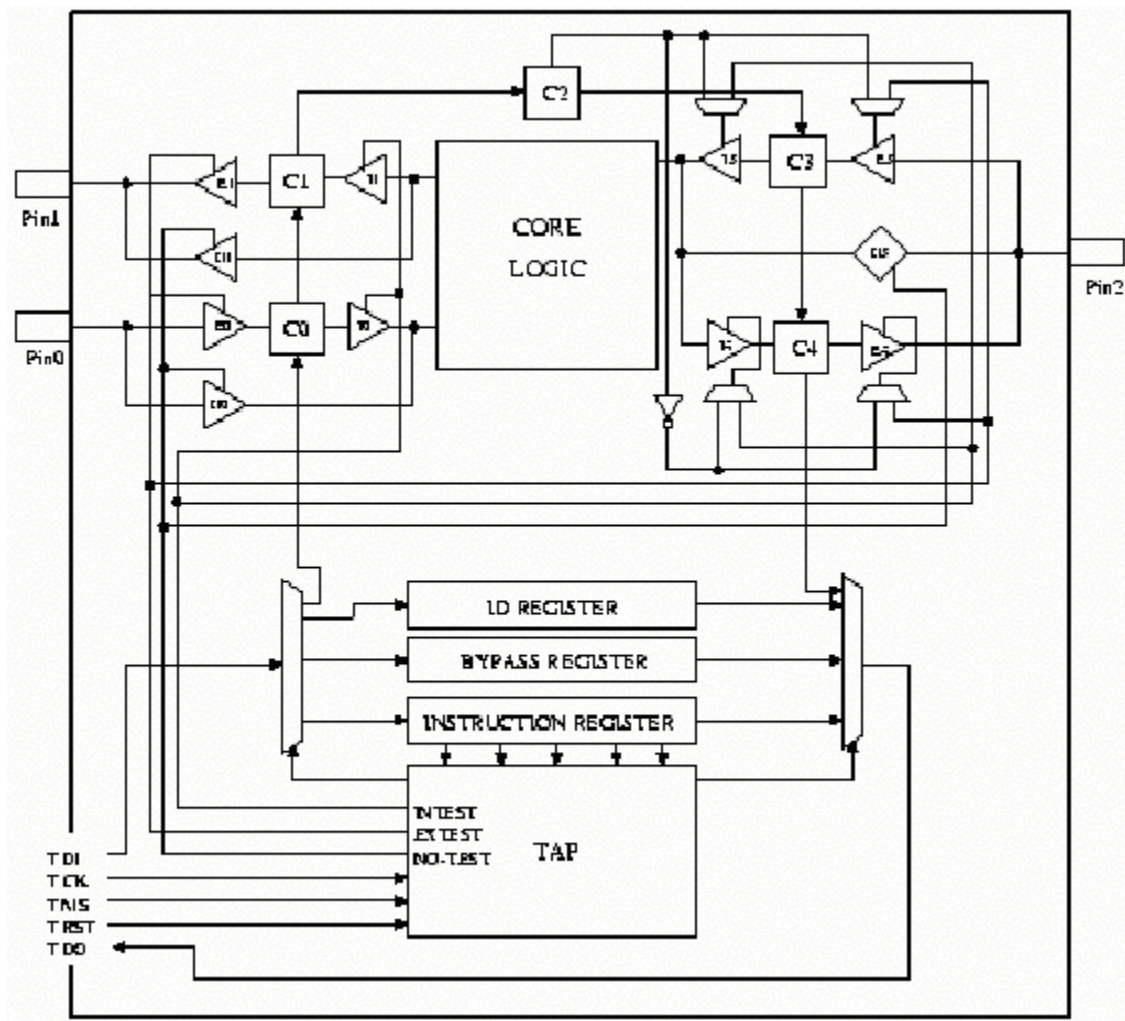
3.4.3 เครื่องมือที่ใช้ในการพัฒนา

การพัฒนาโปรแกรมทั้งหมดของโครงการนี้ใช้ภาษาซี ซึ่งเครื่องมือที่เลือกใช้ในการพัฒนาโปรแกรมบนระบบปฏิบัติการลินุกซ์ เป็นเครื่องมือที่อยู่ในชุด GNU ของ Free Software Foundation คือ binutils gcc และ gdb ซึ่งเครื่องมือทั้งคู่อุปกรณ์ที่จะใช้ทำงานกับโพรเซสเซอร์ในตระกูล ARM7TDMI ได้อยู่แล้ว ทำให้สามารถใช้เป็นเครื่องมือหลักในการพัฒนาโปรแกรมรวมทั้งไดรเวอร์ให้ทำงานบนบอร์ดควบคุมได้

3.4.3.1 JTAG Boundary Scan Interface

ปัจจุบันอุปกรณ์อิเล็กทรอนิกส์มีขนาดเล็กลงมาก ทำให้PCB (Printed – Circuit Boards) ที่นำมาใช้พัฒนามีขนาดเล็กลงด้วย และส่งผลให้เกิดวิธีการพัฒนายูนิตที่เป็นลักษณะ On – Board Programming (OBP) โดยใช้เทคนิคที่เรียกว่า Joint Test Action Group Test Access Port (JTAG TAP)

JTAG TAP เป็นมาตรฐานของ IEEE 1149.1 ซึ่งกล่าวอ้างถึง การทดสอบการทำงานของพอร์ตและสถาปัตยกรรม Boundary – Scan



รูปที่ 3.18 สถาปัตยกรรมของ JTAG Boundary Scan

การทำงาน

สัญญาณทั้งหมดที่อยู่ระหว่าง core logic และเส้นทางเดินของสัญญาณขาของตัวชิพซึ่งเรียกว่า “Boundary Scan Register” (BSR) จากรูปแสดง ด้วยเซลล์ C1, C2, C3, C4 และมีเส้นทางต่อจากขาไปยัง core logic ในการทำงานในโหมด external-test mode ขาจะไม่สามารถต่อถึงเข้ากับ core logic ได้ จากรูปมี ขา Pin1 กับขา Pin2 เป็นเอาต์พุต และจะทำการอ่านค่าแล้วทำการ latch โดยขาอินพุตคือ ขา Pin0 กับขา Pin2 แต่ถ้าเป็นการทำงานใน internal-test mode ขาจะไม่ต่อถึงกับ core logic แต่ core logic จะเป็นตัวรับสัญญาณอินพุต และจะทำการอ่านค่าและ latch สัญญาณเอาต์พุตที่ core logic

จากรูป สมมุติให้ JTAG ทำงานในโหมด external-test mode โดย C0 เป็น BSR เซลล์ แล้วทำการอ่านค่าที่ขาอินพุต Pin0, C1 เป็น BSR เซลล์เอาต์พุตที่ขา Pin1, C2 ไม่มีการตอบสนองการทำงานกับขาใดๆ แต่มันจะ “enable” BSR เซลล์ ซึ่งควบคุมการทำงานแบบทางเดียว จากการทำงานที่เป็นสองทางที่ขา Pin2, C3

เป็นอินพุต BSR แบบสองทางที่ต่อกับขา Pin2 และ C4 เป็นเอาต์พุต BSR เซลล์ แบบสองทางที่ต่อกับขา Pin2 เราสามารถสรุปการทำงานของ BSR เซลล์ได้ 3 อย่าง ดังนี้

- Input Cells คือ C0 , C3 ซึ่งจะทำงานสัมพันธ์กับขาที่ต่ออยู่ในการ Capture เมื่อต่อกับ JTAG ขณะทำงานใน external test mode
- Output Cells คือ C1 , C4 ซึ่งจะทำงานสัมพันธ์กับขาที่ต่ออยู่ในการให้ค่าออกมา ขณะทำงานใน external test mode
- Enable Cells คือ C2 จะไม่ขึ้นอยู่กับขาใดๆ แต่จะควบคุมให้มีการไหลของข้อมูลเพียงทางเดียวหรือว่าสองทาง หรือจะกำหนดให้ขาสัญญาณเป็นอินพุตหรือเอาต์พุต

ส่วนเกท E0 , E3 และ E4 การทำงานจะถูกควบคุมโดย TAP (และอาจควบคุมด้วยการ “enable” เซลล์ เหมือนกับ C2) การอ่านค่าหรือนำค่าไปใช้ จะเป็นไปตามลำดับของสถานะ(State)อินพุตหรือเอาต์พุต , เซลล์ , จาก ขาของตัวชิพ สถานะ(State)ในการอ่านค่า (capture) หรือ การนำค่าไปใช้ (apply) ซึ่งจะมีช่วงเปลี่ยนการทำงานที่แน่นอนในตาม TAP state-machine ซึ่งอาจจะเป็นตอนที่ IR (instruction register) มีการโหลดค่าก่อน แล้วนำมาเก็บ เป็น opcode (EXTEST)

เกท I0,I3 และ I4 การทำงานจะถูกควบคุมโดย TAP (และอาจควบคุมด้วยการ “enable” เซลล์ เหมือนกับ C2) การอ่านค่าหรือนำค่าไปใช้ จะเป็นไปตามลำดับของสถานะ(State)อินพุตหรือเอาต์พุต , เซลล์ , จาก สัญญาณภายในลอจิกของตัวชิพสถานะ(state)ในการอ่านค่า(capture) หรือ การนำค่าไปใช้ (apply) ซึ่งจะมีช่วงเปลี่ยนการทำงานที่แน่นอนใน TAP state-machine state-machine ซึ่งอาจจะเป็นตอนที่ IR (instruction register) มีการโหลดค่าก่อน แล้วนำมาเก็บ เป็น opcode (INTEST).

เกท N0 , N1 และ N3 จะทำงานเมื่อระบบอยู่โหมดปกติ (normal-operation mode) เท่านั้น และขาของชิป ต่อถึงกับ core-logic คำรีจิสเตอร์ของ BSR สามารถเขียนและอ่านได้บิตต่อบิต ตามลำดับ โดยใช้สัญญาณ TDI และ TDO ของ JTAG ในความเป็นจริงในการอ่านหรือเขียน BSR ในช่วงเวลาเดียวกันของค่าใหม่จาก ขาสัญญาณ TDI ขณะที่มีการ shift ค่าออกจากขาสัญญาณ TDO จะใช้เทคนิคเดียวกันในการอ่านและเขียนค่าของ JTAG registers ตัวอื่นๆ ด้วย โดยจะใช้ TAP controller เป็นตัวต่อระหว่างขา TDI , TDO กับตัว BSR

สัญญาณที่ใช้ในการติดต่อ

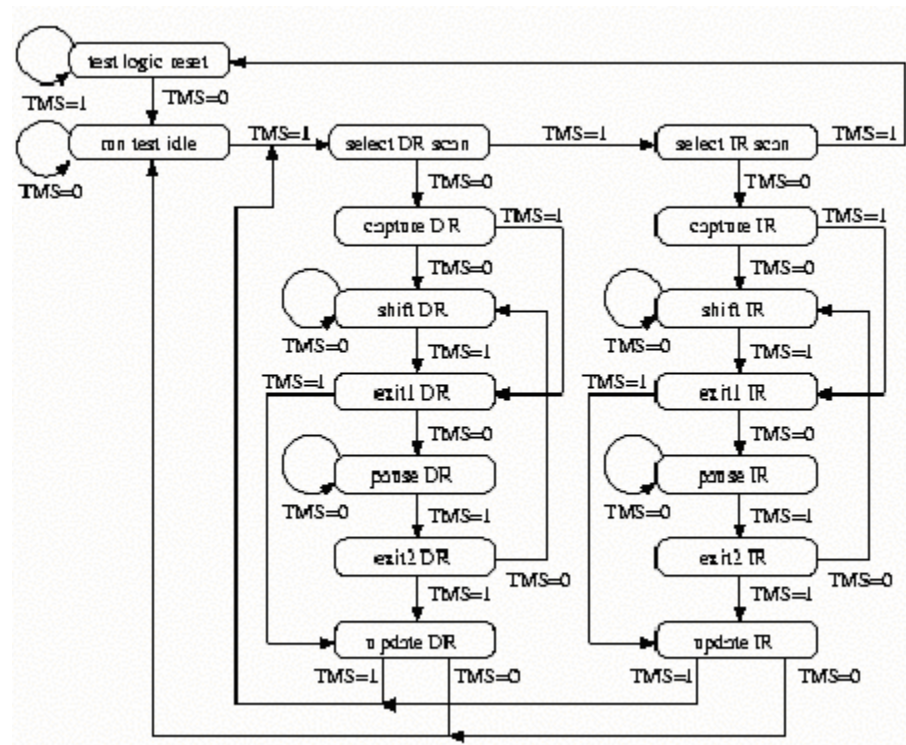
JTAG ประกอบด้วยขาสัญญาณที่ใช้สำหรับควบคุมในการทำงาน 5 ขา คือ

- TRSC - Test Reset Input (active low) เป็นสัญญาณอะซิงโครนัสของ TAP controller ที่ใช้ในช่วงเริ่มต้นการติดต่อและยกเลิกการติดต่อ
- TCK - Test Clock Input เป็นสัญญาณคล็อกจากภายนอกไม่ได้ขึ้นกับระบบ
- TMS - Test Mode Select Input ใช้ควบคุมการเปลี่ยนโหมดการทำงานของ state machine

- TDI - Test Data Input เป็นขาที่ใช้รับข้อมูลไปยัง JTAG register หรือ Boundary Scan chain (Boundary Scan Register, หรือ other data registers อื่นๆ) โดยข้อมูลจะผ่านทางขา TDI
 - TDO - Test Data Output เป็นขาที่ใช้รับข้อมูลออกจาก JTAG register หรือ Boundary Scan chain
- โดยปกติ การทดสอบการทำงานของบอร์ดที่ประกอบขึ้นด้วยจำนวนชิพหลายตัว ที่สนับสนุนการทำงานแบบ JTAG จะต่อสัญญาณ TRST, TCK และ TMS แบบขนานกับชิพทุกตัว และต่อขาสัญญาณ TDO จากชิพตัวหนึ่งไปยัง TDI อีกตัวในลักษณะลูป

TAP controller

เป็นตัวที่ใช้สำหรับควบคุมการติดต่อขาสัญญาณของ JTAG กับ BSR ของชิพ ซึ่งมีขั้นตอนการทำงานตาม state-machine ดังรูปข้างล่าง โดยการเปลี่ยนของสถานะ(State) จะขึ้นกับขาสัญญาณ TMS



รูปที่ 3.19 ภาพแสดงไคอะแกรมของการเปลี่ยนสถานะ

จากภาพไคอะแกรมข้างบน จะเห็นว่าการออกจากสถานะ(State)มีสองทาง การเปลี่ยนของสถานะ(State) จะขึ้นอยู่กับสัญญาณ TMS เพียงสัญญาณเดียว ในไคอะแกรมจะมีสองทางหลักในการเปลี่ยนการทำงานของ (State)ซึ่งเป็นการควบคุมการทำงานของ Data Register (ID register , Bypass register , BSR register) และ Instruction register โดย Data Register จะมีการทำงานทุกครั้งเมื่อทางเดินของ Data Register ถูกเลือกด้วยการโหลดค่าลงใน Instruction Register

การเปลี่ยนเส้นทางตามลำดับของโปรแกรมข้างล่าง (โดยปกติเรียกว่า IR path) เป็นการโหลดค่าใหม่ใน Instruction Register และ อ่านค่าเดิมออก

```
*-> test logic reset
--> run test idle
--> select dr scan
--> select ir scan
--> capture ir
--> shift ir --> ... n times ... --> shift ir
--> exit1 ir
--> update ir
--> run test idle ->*
```

ค่าใหม่จะถูก shift ลงใน Instruction Register จากขาสัญญาณ TDI เป็นจำนวน 1 bit ซึ่งขึ้นอยู่กับสถานะ "shift ir" ทั้งหมด ส่วนค่าเก่าของ Instruction Register ถูก shift ออกทางขาสัญญาณ TDO เป็นจำนวน 1 บิต ซึ่งเป็นการออกจากสถานะ "shift ir"

การเปลี่ยนเส้นทางตามลำดับของโปรแกรมข้างล่าง (โดยปกติเรียกว่า DR path) เป็นการโหลดค่าใหม่ใน Data Register ที่ได้เลือกไว้ในขณะนั้นแล้ว อ่านค่าเดิมออกไป

```
*-> test logic reset
--> run test idle
--> select dr scan
--> capture dr
--> shift dr --> ... n times ... --> shift dr
--> exit1 dr
--> update dr
--> run test idle ->*
```

ค่าใหม่ถูก shift ลงใน Data Register ที่เลือกไว้ในขณะนั้น จากขาสัญญาณ TDI เป็นจำนวนหนึ่งบิต ซึ่งการทำงานขึ้นอยู่กับสถานะ "shift dr" ทั้งหมด ส่วนค่าเก่าจะออกจาก Data Register ที่เลือกไว้ในขณะนั้นจะถูก shift ออกจากขาสัญญาณ TDO เป็นจำนวนหนึ่งบิต ซึ่งเป็นการออกจากสถานะ "shift dr" และค่าของ Instruction Register จะถูก shift ออกไป การ captured ขึ้นอยู่กับสถานะ "capture ir" ค่าของ Data Register ที่เลือกไว้ในขณะนั้น จะถูก shift ออกไป การ captured ขึ้นอยู่กับสถานะ "capture dr" ค่าใหม่จะถูก shift ลงใน

Instruction Register ซึ่งเป็นการนำค่ามาใช้ (และคำสั่งจะเป็นผล) ซึ่งขึ้นอยู่กับสถานะ "update ir" ค่าของ Data Register ที่เลือกไว้ในขณะนั้นจะถูก นำไปใช้ (คือ การส่งข้อมูลออกที่ขาเอาต์พุต ในกรณีของ BSR) ซึ่งขึ้นอยู่กับสถานะ "update dr"

Data register

เมื่อมีการพูดถึงวิธีการทดสอบการทำงาน จะเป็นผลของการเก็บค่าของ instruction register ซึ่งเป็นการเลือกกระหว่างความแตกต่าง ของ data registers ที่ทำงานในช่วง "dr path" ซึ่งต่อไปนี้เป็นารแสดง data-registers ที่ใช้ใน JTAG

- *Device ID register (IDR)* reads-out เป็นการกำหนดหมายเลขให้กับชิปที่นำมาเชื่อมต่อ
- *Bypass register (BR)* 1-cell pass-through register ซึ่งเป็นการต่อ TDI ด้วย TDO กับช่วงเวลาไป 1clock เพื่อให้ง่ายในการติดต่อกับอุปกรณ์อื่นที่อยู่บนแผ่น PCB เดียวกัน
- *Boundary Scan register (BSR)* ซึ่งอธิบายรายละเอียดเกี่ยวกับสัญญาณที่อยู่ระหว่าง core-logic กับขา

Instructions

โดยปกติของการทดสอบด้วย JTAG เป็นการเข้าสู่คำสั่งซึ่ง เป็นลักษณะชนิดของการทดสอบที่จะกระทำต่อไป และ Data Register ถูกใช้ในช่วงที่มีการทดสอบ ค่าภายใน Instruction Register (จากวิธีการทำงานของ TAP ตลอดถึงการใช้ "ir path") และการใช้ Data Register ในการทดสอบ (จากวิธีการทำงานของ TAP ตลอดถึงการใช้ "dr path" เพียงหนึ่งตัวหรือมากกว่า)

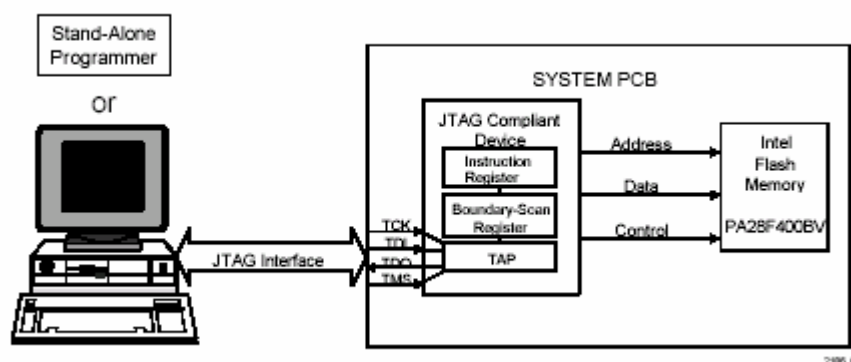
มีคำสั่งที่เป็นคำสั่งเฉพาะ (Private instructions) และ คำสั่งทั่วไป (Public instructions) โดยคำสั่งทั่วไป ได้มาจากเอกสารของโรงงานผู้ผลิตชิพ และใช้กันโดยทั่วไป ส่วนคำสั่งเฉพาะจะไม่เปิดเผย ซึ่งมาตรฐานของ IEEE – 1149 เป็นการกำหนดการใช้ชุดคำสั่งทั่วไปที่แสดงใน JTAG ทั้งหมด โดยข้างล่างนี้เป็นชุดคำสั่งดังกล่าว

- *BYPASS* : นี่เป็นขาสัญญาณ TDI และ TDO ที่ต่อผ่านเข้าไปในรีจิสเตอร์เพียง 1 บิต (single-bit pass-through register) นี่เป็นคำสั่งที่ยอมให้ต่อเข้ากับอุปกรณ์ตัวอื่นในลักษณะที่เหมือนกับบูป
- *EXTEST* : นี่เป็นคำสั่งเกี่ยวกับ boundary scan register (BSR) ที่ต่อระหว่าง สัญญาณ TDI และ TDO มีการ capture ค่าที่ขาของชิพ โดย BSR เซลล์ ซึ่งทั้งหมดจะเป็นการทำงานในสถานะ "capture dr" (ดูการเปลี่ยน สถานะใน diagram) ค่าที่เก็บใน BSR register จะถูก shift ออกไป ผ่านทางขาสัญญาณ TDO แล้วจะออกจากสถานะ "shift dr" ค่าที่เก็บใน BSR register (ที่ได้จากการ capture) จะถูก shift ออกไป แล้วค่าใหม่จะถูก shift เข้ามา ซึ่งการทำงานทั้งหมดจะอยู่ในสถานะ " shift dr" ค่าใหม่ที่เก็บใน BSR และถูกนำมาใช้ โดยเป็นสัญญาณที่ขาของชิพ และจะทำงานในช่วงสถานะ "update dr"
- *IDCODE* : โดย ID register ที่ต่อระหว่าง TDI และ TDO ทั้งหมดจะเป็นการทำงานในสถานะ "capture dr" เป็น Device ID Code (เป็นการเชื่อมต่ออุปกรณ์ โดยมีการกำหนดหมายเลขจากโรงงานผู้ผลิต)

- *INTEST* : นี่เป็นคำสั่งเกี่ยวกับ boundary scan register (BSR) ที่ต่อระหว่าง สัญญาณ TDI และ TDO สัญญาณภายใน core-logic ของชิพ จะถูกcapture โดย BSR เซลล์ ซึ่งการทำงานทั้งหมดจะอยู่ในสถานะ "capture dr" (ดูการเปลี่ยน สถานะในdiagram) ค่าที่เก็บใน BSR register จะถูก shifted ออกผ่านทางขาสัญญาณ TDO ซึ่งจะออกจากสถานะ "shift dr" ค่าที่เก็บใน BSR register (ที่ได้จากการ capture) จะถูก shift ออกไป แล้วค่าใหม่จะถูก shift เข้ามา ซึ่งการทำงานทั้งหมดจะอยู่ในสถานะ "shift dr" ค่าใหม่ที่เก็บใน BSR และถูกนำมาใช้ เป็นสัญญาณให้กับ core-logic ในช่วงที่ทำงานในสถานะ "update dr"

3.4.3.2 On-Board Programming

สิ่งจำเป็นอันดับแรกก่อนที่จะนำโปรแกรมที่พัฒนาไปทำงานยังที่แพลตฟอร์มที่ได้ออกแบบขึ้นจำเป็นต้องมีโปรแกรมที่มีการติดต่อระหว่างเครื่องโฮส(Host)และเครื่องแพลตฟอร์ม(Target) หรือบอร์ดควบคุม(On-Board Programming) ขั้นตอนนี้ได้ทำการพัฒนาเครื่องมือที่ใช้ในการโปรแกรมหน่วยความจำแฟลช (Flasher) ซึ่งพัฒนาต่อจากโปรแกรม “Armttool” โดยจะส่งข้อมูลผ่าน พอร์ตขนานทำการโปรแกรมผ่านพอร์ต JTAG ของไมโครคอนโทรเลอร์ ดังรูปข้างล่างแสดงการทำงานของ On-Board Programming



รูปที่ 3.20 การทำ On-Board Pramming

โปรแกรม “Armttool” จะเป็นมีอินเตอร์เฟซเป็นคอมมานด์ไลน์ทำงานได้ทั้งในระบบปฏิบัติการวินโดวส์ Windows + Cygwin(Cygwin= เป็นโปรแกรมที่จำลองสภาพแวดล้อมของระบบปฏิบัติการยูนิกซ์บนวินโดวส์) และระบบปฏิบัติการลินุกซ์ ดังรูป

```

Administrator@adenalin /ecos-d/flashprog/dflasher
$ ./armtool.exe
Use: ./armtool r{read}  adr words {filename}  ... read from target
     ./armtool w{rite}  adr value              ... write a single word to target
     ./armtool w{rite}  adr words filename    ... write file contents to target
     ./armtool x{ecute} adr                  ... execute program at 'adr'
     ./armtool s              ... reset hardware

Administrator@adenalin /ecos-d/flashprog/dflasher
$

```

รูปที่ 3.21 อินเทอร์เฟซของโปรแกรม Armtool บนวินโดวส์

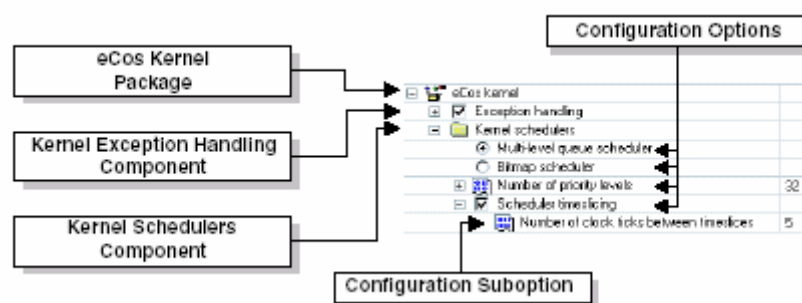
โปรแกรม “Armtool” สามารถที่จะทำงานเป็นตัวโหลด หรือ อ่านข้อมูลของหน่วยความจำ ยกเว้น หน่วยความจำแฟลช ซึ่งต้องเขียนโปรแกรมเพิ่มขึ้นมา คำสั่งที่เป็นประโยชน์อีกคำสั่งคือ คำสั่ง เอกซคิว เพื่อเป็นการสั่งให้ ARM7TDMI ทำงานตามที่ระบุในแอดเดรส สำหรับการโปรแกรมหน่วยความจำแฟลชจะใช้โปรแกรม Flasher ซึ่งอาศัยการทำงานของ Armtool และ Flasher code ที่เขียนขึ้น

3.4.3.3 eCos Configuration tool

eCos Configuration tool เป็นเครื่องมือที่ใช้ในการพัฒนาระบบปฏิบัติการอีคอส(eCos) ซึ่งเป็นตัวที่ใช้ในการปรับ แต่งแก้ไขซอร์สโค้ด ก่อนที่จะทำการรวบรวมเป็นส่วนประกอบหรือคอมโพเนนท์ของไฟล์ตามที่ต้องการ โดยซอร์สโค้ดดังกล่าวจะเก็บรวบรวมไว้ในคอมโพเนนท์ที่ชื่อว่า “*component repository*” คอมไฟล์เพื่อเก็บเป็นไลบรารี สำหรับนำไปลิงค์รวมกับ โปรแกรมแอปพลิเคชันที่เขียนขึ้น

วิธีการของ eCos Configuration

ในระบบฝังตัวต้องมีขนาดเล็ก เร็ว ถูก สามารถทำได้โดยการควบคุมให้มีซอฟต์แวร์ที่จำเป็นอยู่ในระบบเท่านั้น ซึ่งจะแตกต่างจากวิธีการที่กำหนดให้มีคอมโพเนนท์เข้าไปรวมกับแอปพลิเคชัน

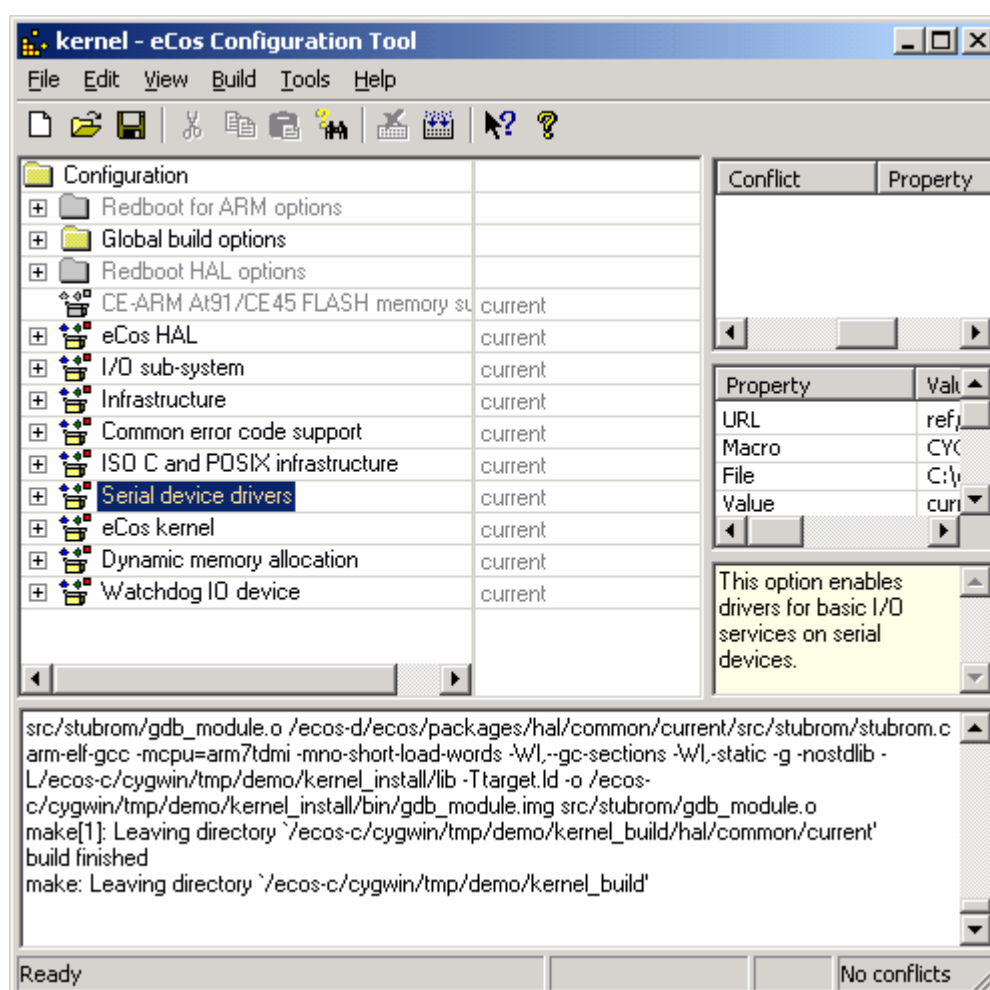


รูปที่ 3.22 การ configuration eCos

การลดขนาดแอปพลิเคชันระบบฝังตัว eCos จะใช้งาน Compile-time control โดยให้นักพัฒนาระบบจัดการกับคอมโพเนนท์ และเรียกใช้คอมโพเนนท์นั้นๆ สำหรับสร้างแอปพลิเคชันตามที่ตั้งใจ ซึ่งจะทำให้ลด

ขนาดของโค้ดได้ดีที่สุด เพราะว่าเป็นการควบคุมที่ระดับ Statement ใน source code มากกว่าที่จะไปควบคุมที่ Function หรือ object การใช้งาน Compile-time จึงเหมาะสำหรับการพัฒนาระบบฝังตัว นอกจากโค้ดจะมีขนาดเล็กแล้วยังมีข้อดีที่สำคัญคือ

- แอปพลิเคชันทำงานได้เร็วกว่าเพราะว่าไม่มีการตรวจสอบตัวแปรในระหว่าง run-time เพื่อหาว่าต้องทำอะไร
- โปรแกรมมีการตอบสนองมากกว่า และ ส่วนที่ซ้ำซ้อนจะถูก reused ซึ่งเป้าหมายในการสร้างระบบตรวจสอบสำหรับ real-time device
- โค้ดที่ง่ายกว่าจะถูกสร้างขึ้น ทำให้ตรวจสอบ และ ทดสอบทำได้ง่ายขึ้น
- ทำให้ลดค่าใช้จ่ายเพราะว่าทรัพยากรที่ใช้จะถูก optimize และการใช้งาน processor cycle อย่างมีประสิทธิภาพ ด้วยเหตุนี้จึงต้องการ hardware ที่มีราคาถูกกว่าในการออกแบบ



รูปที่ 3.23 eCos Configuration tool

3.5 วิธีการที่จะนำโปรแกรมลงไปทำงานบนบอร์ดทำได้ 2 วิธี ดังนี้

3.5.1 การ Burning

คือ การนำเอาโปรแกรมและ Rom Filesystem image ที่ผ่านการคอมไพล์เรียบร้อยแล้วไปใส่ในหน่วยความจำประเภทคงอยู่ (Nonvolatile Memory) เช่น หน่วยความจำแฟลช (Flash Memory)

3.5.2 การ Download

คือการนำเอาโปรแกรมและ Rom Filesystem image ที่ผ่านการคอมไพล์เรียบร้อยแล้วโหลดผ่านทางสายสื่อสารไปใส่ในหน่วยความจำ เช่น พอร์ตขนาน หรือ พอร์ตอนุกรม เป็นต้น โดยการเรียกให้ทำงานให้เริ่มต้นทำงานในตำแหน่งที่ได้โหลดลงไป

ในวิธีแรกนั้นเหมาะสมกับระบบที่ได้มีการพัฒนาเรียบร้อยแล้วและพร้อมที่จะนำไปใช้งาน ดังนั้นจึงได้เลือกใช้วิธีที่สองแทน เพราะในขั้นตอนของการทดลองและทดสอบการทำงานจะต้องมีการแก้ไขอยู่เป็นประจำ รวมทั้งต้องมีการดีบั๊กข้อผิดพลาดต่างๆ อยู่เสมอ ถ้ามีข้อมูลอยู่ในแรมอยู่แล้วจะทำให้สามารถที่จะดึงเอาข้อมูลที่อยู่ในแรมออกมาดูได้อย่างสะดวกยิ่งขึ้น ส่วนการโหลดอิมเมจลงไปทำงานบนบอร์ดนั้น ได้เลือกใช้การโหลดผ่านทางพอร์ตขนานของเครื่องพีซี ซึ่งเชื่อมต่อกับพอร์ต JTAG ของบอร์ด เพราะได้มีการพัฒนาทั้งในส่วนของฮาร์ดแวร์ และซอฟต์แวร์ ที่ใช้การติดต่อกับ JTAG เรียบร้อยอยู่แล้ว

3.6 การพัฒนา eCos สำหรับบอร์ดควบคุม

สิ่งที่ต้องทำเพื่อให้ eCos สามารถทำงานได้กับบอร์ดควบคุมที่ออกแบบขึ้นโดยการปรับปรุงในส่วน of HAL(Hardware Abstraction Layer) โดยการเพิ่มแพลตฟอร์มใหม่เข้าไป ดังนี้

- เพิ่มข้อมูลเกี่ยวกับบอร์ดควบคุมเข้าไปใน Database ให้กับ eCos configuration tool
- เพิ่มรายละเอียดของ Memory Layout
- เพิ่มรายละเอียดของ Memory Controller Initialize
- เพิ่มรายละเอียดของ Interrupt Controller Handling
- เพิ่ม Serial Port device driver สำหรับ GDB และ โปรแกรมสำหรับตรวจสอบ
- เพิ่ม System Timer Initialize และ Control
- เพิ่ม Wallclock driver

หลังจากได้ทำการพัฒนาแพลตฟอร์มใหม่เข้าไป ชื่อ “CE Control board DEMO (CE45)” โดยการปรับปรุงแพลตฟอร์มที่ใกล้เคียงกันคือ ATMEL EB40 ได้ผลเป็นดังรูป (เลือก Template เมนูของ eCos Configuration tool) จากนั้นใช้ eCos Configuration tool ทำการปรับปรุงส่วนของ option ที่ต้องการสำหรับแอปพลิเคชันระบบฝัง ทำการคอมไพล์ซึ่งจะได้เป็นไบนารีเพื่อนำไปลิงก์กับแอปพลิเคชันที่เขียนขึ้น

ตัวอย่างของโปรแกรมแอปพลิเคชัน จะถูกเรียกใช้จากฟังก์ชัน `cyg_user_start()`

```
/* File Application.c */
```

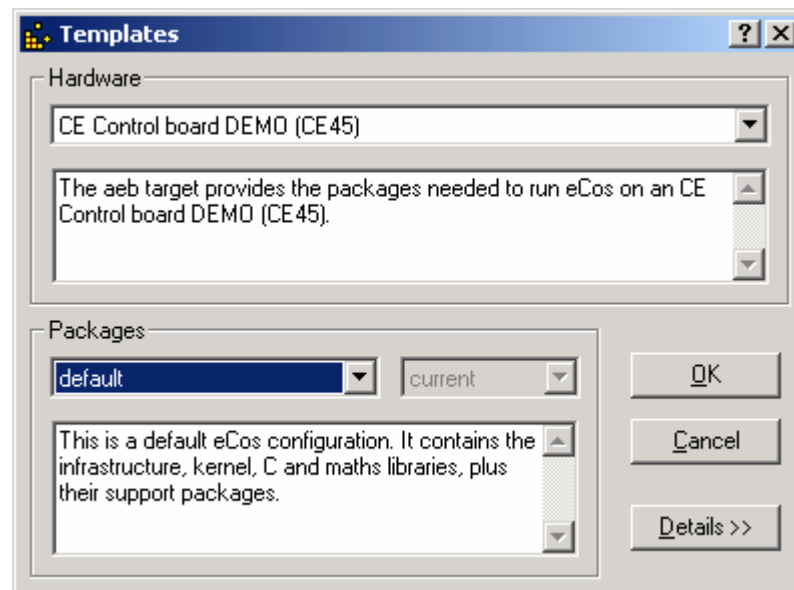
```
void cyg_user_start()
```

```
{
```

```
    printf("Hello eCos World with CE Control Board DEMO(CE45)");
```

```
    /* To DO โค้ด สำหรับ แอปพลิเคชัน */
```

```
}
```



รูปที่ 3.24 แสดงแพลตฟอร์ม “CE Control Board DEMO (CE45)”